Dynamic system virtual address space management

Thirty-two–bit versions of Windows manage the system address space through an internal kernel virtual allocator mechanism, described in this section. Currently, 64-bit versions of Windows have no need to use the allocator for virtual address space management (and thus bypass the cost) because each region is statically defined (refer to **Figure 5-13**).

When the system initializes, the `MiInitializeDynamicVa` function sets up the basic dynamic ranges and sets the available virtual address to all available kernel space. It then initializes the address space ranges for boot loader images, process space (hyperspace), and the HAL through the `MiInitialize-SystemVaRange` function, which is used to set hard-coded address ranges (on 32-bit systems only). Later, when non-paged pool is initialized, this function is used again to reserve the virtual address ranges for it. Finally, whenever a driver loads, the address range is relabeled to a driver-image range instead of a boot-loaded range.

After this point, the rest of the system virtual address space can be dynamically requested and released through `MiObtainSystemVa` (and its analogous `MiObtainSessionVa`) and `MiReturnSystemVa`. Operations such as expanding the system cache, the system PTEs, non-paged pool, paged pool, and/or special pool; mapping memory with large pages; creating the PFN database; and creating a new session all result in dynamic virtual address allocations for a specific range.

Each time the kernel virtual address space allocator obtains virtual memory ranges for use by a certain type of virtual address, it updates the `MiSystemVaType` array, which contains the virtual address type for the newly allocated range. The values that can appear in `MiSystemVaType` are shown in **Table 5-8**.

Region	Description	Limitable
MiVaUnused (0)	Unused	N/A
MiVaSessionSpace (1)	Addresses for session space	Yes
MiVaProcessSpace (2)	Addresses for process address space	No
MiVaBootLoaded (3)	Addresses for images loaded by the boot loader	No
MiVaPfnDatabase (4)	Addresses for the PFN database	No
MiVaNonPagedPool (5)	Addresses for the non-paged pool	Yes
MiVaPagedPool (6)	Addresses for the paged pool	Yes
MiVaSpecialPoolPaged (7)	Addresses for the special pool (paged)	No
MiVaSystemCache (8)	Addresses for the system cache	No
MiVaSystemPtes (9)	Addresses for system PTEs	Yes
MiVaHal (10)	Addresses for the HAL	No
MiVaSessionGlobalSpace (11)	Addresses for session global space	No
MiVaDriverImages (12)	Addresses for loaded driver images	No
MiVaSpecialPoolNonPaged (13)	Addresses for the special pool (non-paged)	Yes
MiVaSystemPtesLarge (14)	Addresses for large page PTEs	Yes

TABLE 5-8 System virtual address types

Although the ability to dynamically reserve virtual address space on demand allows better management of virtual memory, it would be useless without the ability to free this memory. As such, when the paged pool or system cache can be shrunk, or when special pool and large page mappings are freed, the associated virtual address is freed. Another case is when the boot registry is released. This allows dynamic management of memory depending on each component's use. Additionally, components can reclaim memory through `MiReclaimSystemVa`, which requests virtual addresses associated with the system cache to be flushed out (through the dereference segment thread) if available virtual address space has dropped below 128 MB. Reclaiming can also be satisfied if initial non-paged pool has been freed.

In addition to better proportioning and better management of virtual addresses dedicated to different kernel memory consumers, the dynamic virtual address allocator also has advantages when it comes to memory footprint reduction. Instead of having to manually pre-allocate static page table entries and page tables, paging-related structures are allocated on demand. On both 32-bit and 64-bit systems, this reduces boot-time memory usage because unused addresses won't have their page tables allocated. It also means that on 64-bit systems, the large ad-

dress space regions that are reserved don’t need to have their page tables mapped in memory. This allows them to have arbitrarily large limits, especially on systems that have little physical RAM to back the resulting paging structures.

EXPERIMENT: QUERYING SYSTEM VIRTUAL ADDRESS USAGE (WINDOWS 10 AND SERVER 2016)

You can look at the current and peak usage of each system virtual address type by using the kernel debugger. The global variable `MiVisibleState` (of type `MI_VISIBLE_STATE`) provides information available in the public symbols. (The example is on x86 Windows 10.)

1. To get a sense of the data provided by `MiVisibleState`, dump the structure with values:

Click here to view code image

```
lkd> dt nt!_mi_visible_state poi(nt!MiVisibleState)
   +0x000 SpecialPool      : _MI_SPECIAL_POOL
   +0x048 SessionWsList    : _LIST_ENTRY [ 0x91364060 - 0x9a172060 ]
   +0x050 SessionIdBitmap  : 0x8220c3a0 _RTL_BITMAP
   +0x054 PagedPoolInfo    : _MM_PAGED_POOL_INFO
   +0x070 MaximumNonPagedPoolInPages : 0x80000
   +0x074 SizeOfPagedPoolInPages : 0x7fc00
   +0x078 SystemPteInfo    : _MI_SYSTEM_PTE_TYPE
   +0x0b0 NonPagedPoolCommit : 0x3272
   +0x0b4 BootCommit       : 0x186d
   +0x0b8 MdlPagesAllocated : 0x105
   +0x0bc SystemPageTableCommit : 0x1e1
   +0x0c0 SpecialPagesInUse : 0
   +0x0c4 WsOverheadPages  : 0x775
   +0x0c8 VadBitmapPages   : 0x30
   +0x0cc ProcessCommit    : 0xb40
   +0x0d0 SharedCommit     : 0x712a
   +0x0d4 DriverCommit     : 0n7276
   +0x100 SystemWs         : [3] _MMSUPPORT_FULL
   +0x2c0 SystemCacheShared : _MMSUPPORT_SHARED
   +0x2e4 MapCacheFailures : 0
   +0x2e8 PagefileHashPages : 0x30
   +0x2ec PteHeader        : _SYSPTES_HEADER
   +0x378 SessionSpecialPool : 0x95201f48 _MI_SPECIAL_POOL
   +0x37c SystemVaTypeCount : [15] 0
```

+0x3b8 SystemVaType : [1024] ""
+0x7b8 SystemVaTypeCountFailures : [15] 0
+0x7f4 SystemVaTypeCountLimit : [15] 0
+0x830 SystemVaTypeCountPeak : [15] 0
+0x86c SystemAvailableVa : 0x38800000

2. Notice the last arrays with 15 elements each, corresponding to the system virtual address types from **Table 5-8**. Here are the `SystemVaTypeCount` and `SystemVaTypeCountPeak` arrays:

Click here to view code image

```
lkd> dt nt!_mi_visible_state poi(nt!mivisiblestate) -a
SystemVaTypeCount
   +0x37c SystemVaTypeCount :
    [00] 0
    [01] 0x1c
    [02] 0xb
    [03] 0x15
    [04] 0xf
    [05] 0x1b
    [06] 0x46
    [07] 0
    [08] 0x125
    [09] 0x38
    [10] 2
    [11] 0xb
    [12] 0x19
    [13] 0
    [14] 0xd
lkd> dt nt!_mi_visible_state poi(nt!mivisiblestate) -a
SystemVaTypeCountPeak
   +0x830 SystemVaTypeCountPeak :
    [00] 0
    [01] 0x1f
    [02] 0
    [03] 0x1f
```

[04] 0xf
[05] 0x1d
[06] 0x51
[07] 0
[08] 0x1e6
[09] 0x55
[10] 0
[11] 0xb
[12] 0x5d
[13] 0
[14] 0xe

EXPERIMENT: QUERYING SYSTEM VIRTUAL ADDRESS USAGE (WINDOWS 8.*X* AND SERVER 2012/R2)

You can look at the current and peak usage of each system virtual address type by using the kernel debugger. For each system virtual address type described in **Table 5-8**, the `MiSystemVa-TypeCount`, `MiSystemVaTypeCountFailures`, and `MiSystemVaTypeCountPeak` global arrays in the kernel contain the sizes, count failures, and peak sizes for each type. The size is in multiples of a PDE mapping (see the "**Address translation**" section later in this chapter), which is effectively the size of a large page (2 MB on x86). Here's how you can dump the usage for the system, followed by the peak usage. You can use a similar technique for the failure counts. (The example is from a 32-bit Windows 8.1 system.)

Click here to view code image

```
lkd> dd /c 1 MiSystemVaTypeCount L f
81c16640  00000000
81c16644  0000001e
81c16648  0000000b
81c1664c  00000018
81c16650  0000000f
81c16654  00000017
81c16658  0000005f
```

```
81c1665c  00000000
81c16660  000000c7
81c16664  00000021
81c16668  00000002
81c1666c  00000008
81c16670  0000001c
81c16674  00000000
81c16678  0000000b
lkd> dd /c 1 MiSystemVaTypeCountPeak L f
81c16b60  00000000
81c16b64  00000021
81c16b68  00000000
81c16b6c  00000022
81c16b70  0000000f
81c16b74  0000001e
81c16b78  0000007e
81c16b7c  00000000
81c16b80  000000e3
81c16b84  00000027
81c16b88  00000000
81c16b8c  00000008
81c16b90  00000059
81c16b94  00000000
81c16b98  0000000b
```

Theoretically, the different virtual address ranges assigned to components can grow arbitrarily in size if enough system virtual address space is available. In practice, on 32-bit systems, the kernel allocator implements the ability to set limits on each virtual address type for the purposes of both reliability and stability. (On 64-bit systems, kernel address space exhaustion is currently not a concern.) Although no limits are imposed by default, system administrators can use the registry to modify these limits for the virtual address types that are currently marked as limitable (see **Table 5-8**).

If the current request during the `MiObtainSystemVa` call exceeds the available limit, a failure is marked (see the previous experiment) and a reclaim operation is requested regardless of available memory. This should help alleviate memory load and might allow the virtual address allocation to work during the next attempt. Recall, however, that reclaiming affects only system cache and non-paged pool.

EXPERIMENT: SETTING SYSTEM VIRTUAL ADDRESS LIMITS

The `MiSystemVaTypeCountLimit` array contains limitations for system virtual address usage that can be set for each type. Currently, the memory manager allows only certain virtual address types to be limited, and it provides the ability to use an undocumented system call to set limits for the system dynamically during run time. (These limits can also be set through the registry, as described at **http://msdn.microsoft.com/en-us/library/bb870880.aspx**.) These limits can be set for those types marked in **Table 5-8**.

You can use the MemLimit utility (found in this book's downloadable resources) on 32-bit systems to query and set the different limits for these types and to see the current and peak virtual address space usage. Here's how you can query the current limits with the `-q` flag:

Click here to view code image

```
C:\Tools>MemLimit.exe -q

MemLimit v1.01 - Query and set hard limits on system VA space consumption
Copyright (C) 2008-2016 by Alex Ionescu
www.alex-ionescu.com

System Va Consumption:

Type              Current         Peak         Limit
Non Paged Pool      45056 KB      55296 KB       0 KB
Paged Pool         151552 KB     165888 KB       0 KB
System Cache       446464 KB     479232 KB       0 KB
System PTEs         90112 KB     135168 KB       0 KB
Session Space       63488 KB      73728 KB       0 KB
```

As an experiment, use the following command to set a limit of 100 MB for paged pool:

```
memlimit.exe -p 100M
```

Now use the Sysinternals TestLimit tool to create as many handles as possible. Normally, with enough paged pool, the number should be around 16 million. But with the limit to 100 MB, it's less:

Click here to view code image

```
C:\Tools\Sysinternals>Testlimit.exe -h

Testlimit v5.24 - test Windows limits
Copyright (C) 2012-2015 Mark Russinovich
Sysinternals - www.sysinternals.com

Process ID: 4780

Creating handles...
Created 10727844 handles. Lasterror: 1450
```

See Chapter 8 in Part 2 for more information about objects, handles, and page-pool consumption.

System virtual address space quotas

The system virtual address space limits described in the previous section allow for the limiting of system-wide virtual address space usage of certain kernel components. However, they work only on 32-bit systems when applied to the system as a whole. To address more specific quota requirements that system administrators might have, the memory manager collaborates with the process manager to enforce either system-wide or user-specific quotas for each process.

You can configure the `PagedPoolQuota`, `NonPagedPoolQuota`, `PagingFileQuota`, and `WorkingSetPagesQuota` values in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management key to specify how much memory of each type a given process can use. This information is read at initialization, and the default system quota block is generated and then assigned to all system

processes. (User processes get a copy of the default system quota block unless per-user quotas have been configured, as explained next.)

To enable per-user quotas, you can create subkeys under the HKLM\SYSTEM\CurrentControlSet\Session Manager\Quota System registry key, each one representing a given user SID. The values mentioned previously can then be created under this specific SID subkey, enforcing the limits only for the processes created by that user. **Table 5-9** shows how to configure these values (which can be done at run time or not) and which privileges are required.

Value Name	Description	Value Type	Dynamic	Privilege
PagedPoolQuota	This is the maximum size of paged pool that can be allocated by this process.	Size in MB	Only for processes running with the system token	SeIncreaseQuotaPrivilege
NonPagedPoolQuota	This is the maximum size of nonpaged pool that can be allocated by this process.	Size in MB	Only for processes running with the system token	SeIncreaseQuotaPrivilege
PagingFileQuota	This is the maximum number of pages that a process can have backed by the page file.	Pages	Only for processes running with the system token	SeIncreaseQuotaPrivilege
WorkingSetPagesQuota	This is the maximum number of pages that a process can have in its working set (in physical memory).	Pages	Yes	SeIncreaseBasePriorityPrivilege unless operation is a purge request

TABLE 5-9 Process quota types

User address space layout

Just as address space in the kernel is dynamic, the user address space is also built dynamically. The addresses of the thread stacks, process heaps, and loaded images (such as DLLs and an application's executable) are dynamically computed (if the application and its images support it) through the ASLR mechanism.

At the operating system level, user address space is divided into a few well-defined regions of memory, as shown in **Figure 5-14**. The executable and DLLs themselves are present as memory-mapped image files, followed by the heap(s) of the process and the stack(s) of its thread(s). Apart from these regions (and some reserved system struc-

tures such as the TEBs and PEB), all other memory allocations are run-time dependent and generated. ASLR is involved with the location of all these run time–dependent regions and, combined with DEP, provides a mechanism for making remote exploitation of a system through memory manipulation harder to achieve. Because Windows code and data are placed at dynamic locations, an attacker cannot typically hard-code a meaningful offset into either a program or a system-supplied DLL.

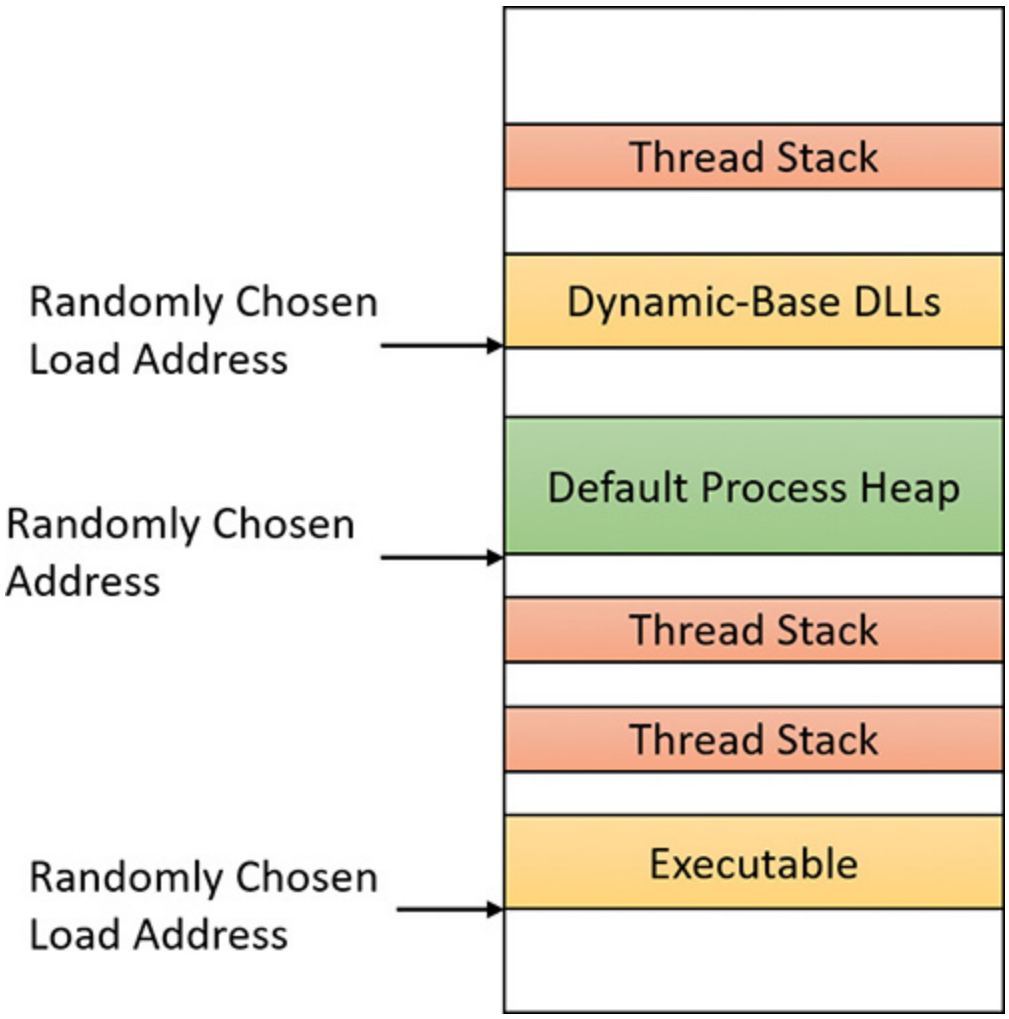


FIGURE 5-14 User address space layout with ASLR enabled.

EXPERIMENT: ANALYZING USER VIRTUAL ADDRESS SPACE

The VMMap utility from Sysinternals can show you a detailed view of the virtual memory being used by any process on your machine. This information is divided into categories for each type of allocation, summarized as follows:

- **Image** This displays memory allocations used to map the executable and its dependencies (such as dynamic libraries) and any other memory-mapped image (portable executable format) files.

- **Mapped File** This displays memory allocations for memory mapped data files.

- **Shareable** This displays memory allocations marked as shareable, typically including shared memory (but not memory-mapped files, which are listed under either Image or Mapped File).

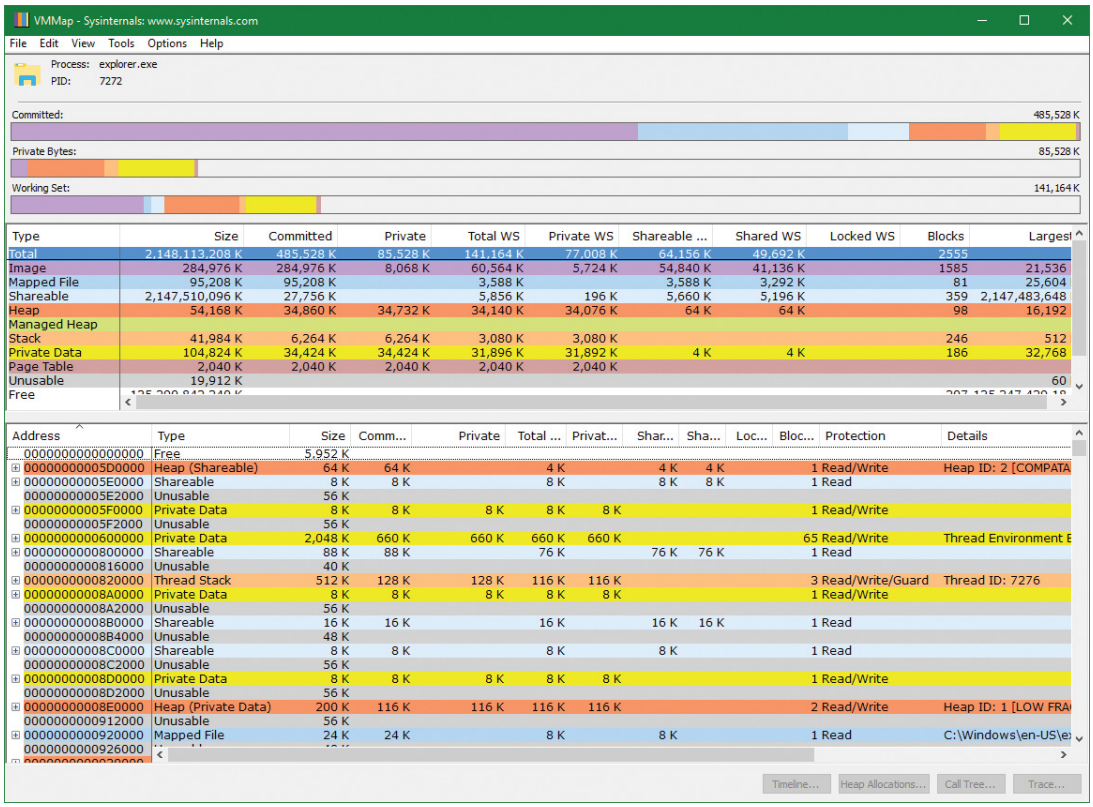
- **Heap** This displays memory allocated for the heap(s) that this process owns.

- **Managed Heap** This displays memory allocated by the .NET CLR (managed objects). It would show nothing for a process that does not use .NET.

- **Stack** This displays memory allocated for the stack of each thread in this process.

- **Private Data** This displays memory allocations marked as private other than the stack and heap, such as internal data structures.

The following shows a typical view of Explorer (64 bit) as seen through VMMap:



Depending on the type of memory allocation, VMMap can show additional information such as file names (for mapped files), heap IDs and types (for heap allocations), and thread IDs (for stack allocations). Furthermore, each allocation's cost is shown both in committed memory and working set memory. The size and protection of each allocation is also displayed.

ASLR begins at the image level, with the executable for the process and its dependent DLLs. Any image file that has specified ASLR support in its PE header (`IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE`), typically specified by using the `/DYNAMICBASE` linker flag in Microsoft Visual Studio, and contains a relocation section will be processed by ASLR. When such an image is found, the system selects an image offset valid globally for the current boot. This offset is selected from a bucket of 256 values, all of which are 64 KB aligned.

Image randomization

For executables, the load offset is calculated by computing a delta value each time an executable is loaded. This value is a pseudo-random 8-bit number from 0x10000 to 0xFE0000, calculated by taking the current processor's time stamp counter (TSC), shifting it by four places, and then performing a division modulo 254 and adding 1. This number is then multiplied by the allocation granularity of 64 KB discussed earlier. By adding 1, the memory manager ensures that the value can never be 0, so executables will never load at the address in the PE header if ASLR is being used. This delta is then added to the executable's preferred load address, creating one of 256 possible locations within 16 MB of the image address in the PE header.

For DLLs, computing the load offset begins with a per-boot, system-wide value called the *image bias*. This is computed by `MiInitializeRelocations` and stored in the global memory state structure (`MI_SYSTEM_INFORMATION`) in the `MiState.Sections.ImageBias` fields (`MiImageBias` global variable in Windows 8.*x*/2012/R2). This value corresponds to the TSC of the current CPU when this function was called during the boot cycle, shifted and masked into an 8-bit value. This provides 256 possible values on 32 bit systems; similar computations are done for 64-bit systems with more possible values as the address space is vast. Unlike executables, this value is computed only once per boot and shared across the system to allow DLLs to remain shared in physical memory and relocated only once. If DLLs were remapped at different locations inside different processes, the code could not be shared. The loader would have to fix up address references differently for each process, thus turning what had been shareable read-only code into process-private data. Each process using a given DLL would have to have its own private copy of the DLL in physical memory.

Once the offset is computed, the memory manager initializes a bitmap called `ImageBitMap` (`MiImageBitMap` global variable in Windows 8.*x*/2012/R2), which is part of the `MI_SECTION_STATE` structure. This bitmap is used to represent ranges from 0x50000000 to 0x78000000 for 32-

bit systems (see the numbers for 64-bit systems below), and each bit represents one unit of allocation (64 KB, as mentioned). Whenever the memory manager loads a DLL, the appropriate bit is set to mark its location in the system. When the same DLL is loaded again, the memory manager shares its section object with the already relocated information.

As each DLL is loaded, the system scans the bitmap from top to bottom for free bits. The `ImageBias` value computed earlier is used as a start index from the top to randomize the load across different boots as suggested. Because the bitmap will be entirely empty when the first DLL (which is always Ntdll.dll) is loaded, its load address can easily be calculated. (Sixty-four–bit systems have their own bias.)

- **32 bit** *0x78000000 – (* `ImageBias` *+* `NtDllSizein64KBChunks` *) * 0x10000*

- **64 bit** *0x7FFFFFFF0000 – (* `ImageBias64High` *+* `NtDllSizein64KBChunks` *) * 0x10000*

Each subsequent DLL will then load in a 64 KB chunk below. Because of this, if the address of Ntdll.dll is known, the addresses of other DLLs could easily be computed. To mitigate this possibility, the order in which known DLLs are mapped by the Session Manager during initialization is also randomized when Smss.exe loads.

Finally, if no free space is available in the bitmap (which would mean that most of the region defined for ASLR is in use), the DLL relocation code defaults back to the executable case, loading the DLL at a 64 KB chunk within 16 MB of its preferred base address.

EXPERIMENT: CALCULATING THE LOAD ADDRESS OF NTDLL.DLL

With what you learned in the previous section, you can calculate the load address of Ntdll.dll with the kernel variable information. The following calculation is done on a Windows 10 x86 system:

1. Start local kernel debugging.

2. Find the `ImageBias` value:

Click here to view code image

```
lkd> ? nt!mistate
Evaluate expression: -2113373760 = 820879c0
lkd> dt nt!_mi_system_information sections.imagebias 820879c0
   +0x500 Sections         :
      +0x0dc ImageBias         : 0x6e
```

3. Open Explorer and find the size of Ntdll.dll in the System32 directory. On this system, it's 1547 KB = 0x182c00, so the size in 64 KB chunks is 0x19 (always rounding up). The result is 0x78000000 – (0x6E + 0x19) * 0x10000 = 0x77790000.

4. Open Process Explorer, find any process, and look at the load address (in the Base or Image Base columns) of Ntdll.dll. You should see the same value.

5. Try to do the same for a 64-bit system.

Stack randomization

The next step in ASLR is to randomize the location of the initial thread's stack and, subsequently, of each new thread. This randomization is enabled unless the `StackRandomizationDisabled` flag was enabled for the process, and consists of first selecting one of 32 possible stack locations separated by either 64 KB or 256 KB. You select this base address by finding the first appropriate free memory region and then choosing

the *x*th available region, where *x* is once again generated based on the current processor's TSC shifted and masked into a 5-bit value. (This allows for 32 possible locations.)
When this base address has been selected, a new TSC-derived value is calculated—this one 9 bits long. The value is then multiplied by 4 to maintain alignment, which means it can be as large as 2,048 bytes (half a page). It is added to the base address to obtain the final stack base.

Heap randomization

ASLR randomizes the location of the initial process heap and subsequent heaps when created in user mode. The `RtlCreateHeap` function uses another pseudo-random TSC-derived value to determine the base address of the heap. This value, 5 bits this time, is multiplied by 64 KB to generate the final base address, starting at 0, giving a possible range of 0x00000000 to 0x001F0000 for the initial heap. Additionally, the range before the heap base address is manually deallocated to force an access violation if an attack is doing a brute-force sweep of the entire possible heap address range.

ASLR in kernel address space

ASLR is also active in kernel address space. There are 64 possible load addresses for 32-bit drivers and 256 for 64-bit drivers. Relocating user-space images requires a significant amount of work area in kernel space, but if kernel space is tight, ASLR can use the user-mode address space of the System process for this work area. On Windows 10 (version 1607) and Server 2016, ASLR is implemented for most system memory regions, such as paged and non-paged pools, system cache, page tables, and the PFN database (initialized by `MiAssignTopLevelRanges`).

Controlling security mitigations

As you've seen, ASLR and many other security mitigations in Windows are optional because of their potential compatibility effects: ASLR applies only to images with the `IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE` bit in their image headers, hardware no-execute (DEP) can be controlled by a combination of boot options and linker options, and so on. To allow both enterprise customers and individual users more visibility and control of these features, Microsoft publishes the Enhanced Mitigation Experience Toolkit (EMET). EMET offers centralized control of the mitigations built into Windows and adds several more mitigations not yet part of the Windows product. Additionally, EMET provides notification capabilities through the event log to let administrators know when certain software has experienced access faults because mitigations have been applied. Finally, EMET enables manual opt-out for certain applications that might exhibit compatibility issues in certain environments, even though they were opted in by the developer.

NOTE

EMET is in version 5.51 at the time of this writing. Its end of support has been extended to the end of July 2018. However, some of its features are integrated in current Windows versions.

EXPERIMENT: LOOKING AT ASLR PROTECTION ON PROCESSES

You can use Process Explorer from Sysinternals to look over your processes (and, just as important, the DLLs they load) to see if they support ASLR. Even if just one DLL loaded by a process does not support ASLR, it can make the process much more vulnerable to attacks.

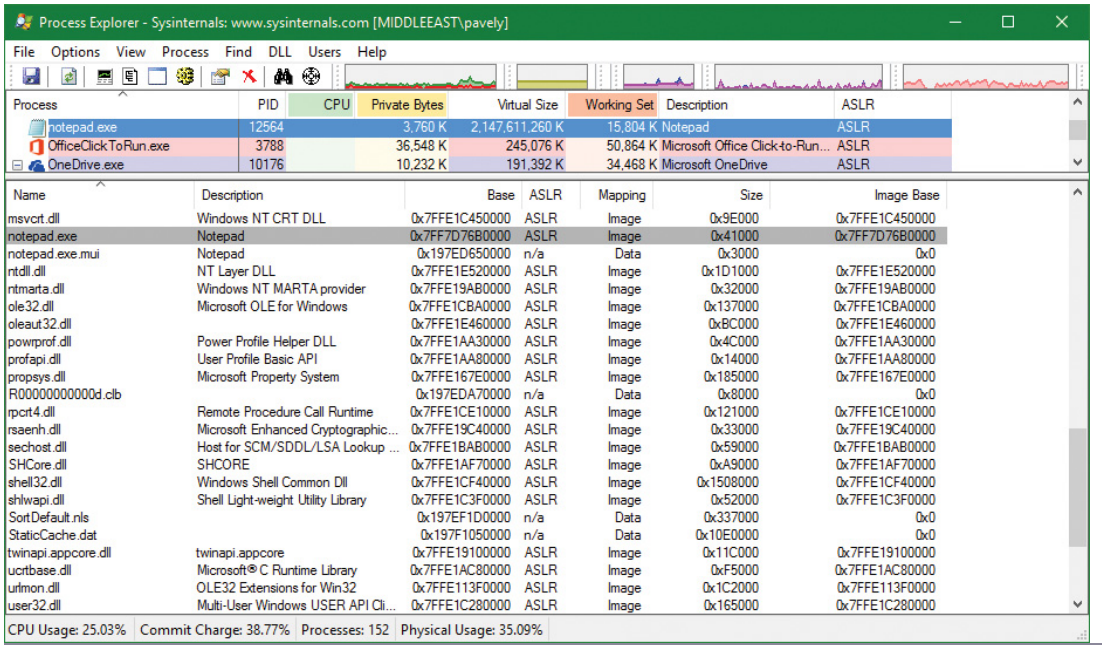
To look at the ASLR status for processes, follow these steps:

1. Right-click any column in the process tree and choose **Select Columns**.

2. Select **ASLR Enabled** on the Process Image and DLL tabs.

3. Notice that all in-box Windows programs and services are running with ASLR enabled, but third-party applications may or may not run with ASLR.

In the example, we have highlighted the Notepad.exe process. In this case, its load address is 0x7FF7D76B0000. If you were to close all instances of Notepad and then start another, you would find it at a different load address. If you shut down and reboot the system and then try the experiment again, you will find that the ASLR-enabled DLLs are at different load addresses after each boot.



Address translation

Now that you've seen how Windows structures the virtual address space, let's look at how it maps these address spaces to real physical pages. User applications and system code reference virtual addresses. This section starts with a detailed description of 32-bit x86 address translation in PAE mode (the only mode supported in recent versions of Windows) and continues with a description of the differences on the ARM and x64 platforms. The next section describes what happens when such a translation doesn't resolve to a physical memory address (page fault) and explains how Windows manages physical memory via working sets and the page frame database.

x86 virtual address translation

The original x86 kernel supported no more than 4 GB of physical memory, based on the CPU hardware available at the time. The Intel x86 Pentium Pro processor introduced a memory-mapping mode called *Physical Address Extension* (*PAE*). With the proper chipset, the PAE mode allows 32-bit operating systems access to up to 64 GB of physical memory on current Intel x86 processors (up from 4 GB without PAE) and up to 1,024 GB of physical memory when running on x64 processors in legacy mode (although Windows currently limits this to 64 GB due to the size of the PFN database required to describe so much memory). Since then, Windows has maintained two separate x86 kernels—one that did not support PAE and one that did. Starting with Windows Vista, an x86 Windows installation always installs the PAE kernel even if the system's physical memory is not higher than 4 GB. This allows Microsoft to maintain a single x86 kernel, as the benefits of the non-PAE kernel in terms of performance and memory footprint became negligible (and is required for hardware no-execute support). Thus, we'll describe only x86 PAE address translation. Interested readers can read the relevant section in the sixth edition of this book for the non-PAE case.

Using data structures that the memory manager creates and maintains called *page tables*, the CPU translates virtual addresses into physical addresses. Each page of virtual address space is associated with a system-space structure called a *page table entry* (*PTE*), which contains the physical address to which the virtual one is mapped. For example, **Figure 5-15** shows how three consecutive virtual pages might be mapped to three physically discontiguous pages on an x86 system. There may not even be any PTEs for regions that have been marked as reserved or committed but never accessed, because the page table itself might be allocated only when the first page fault occurs. (The dashed line connecting the virtual pages to the PTEs in **Figure 5-15** represents the indirect relationship between virtual pages and physical memory.)

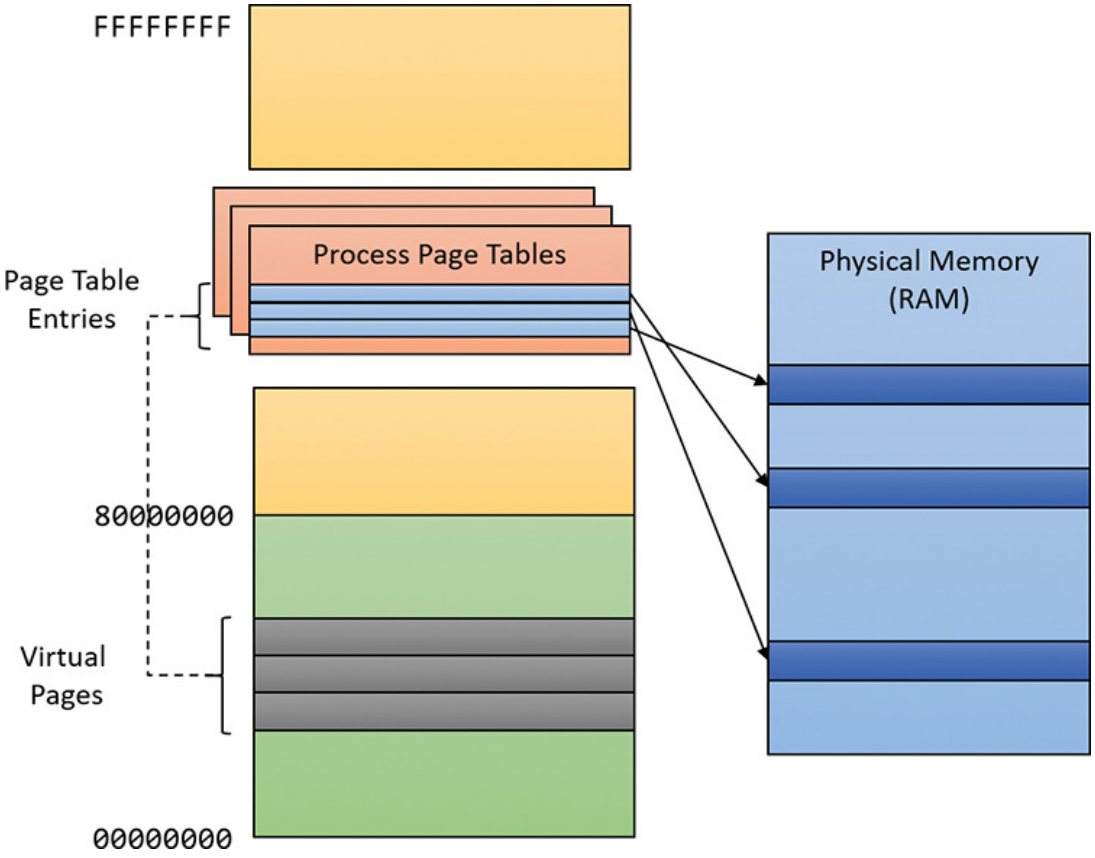


FIGURE 5-15 Mapping virtual addresses to physical memory (x86).

NOTE

Even kernel-mode code (such as device drivers) cannot reference physical memory addresses directly, but it may do so indirectly by first creating virtual addresses mapped to them. For more information, see the memory descriptor list (MDL) support routines described in the WDK documentation.

The actual translation process and the layout of the page tables and page directories (described shortly), are determined by the CPU. The operating system must follow suit and build the structures correctly in memory for the whole concept to work. **Figure 5-16** depicts a general diagram of x86 translation. The general concept, however, is the same for other architectures.

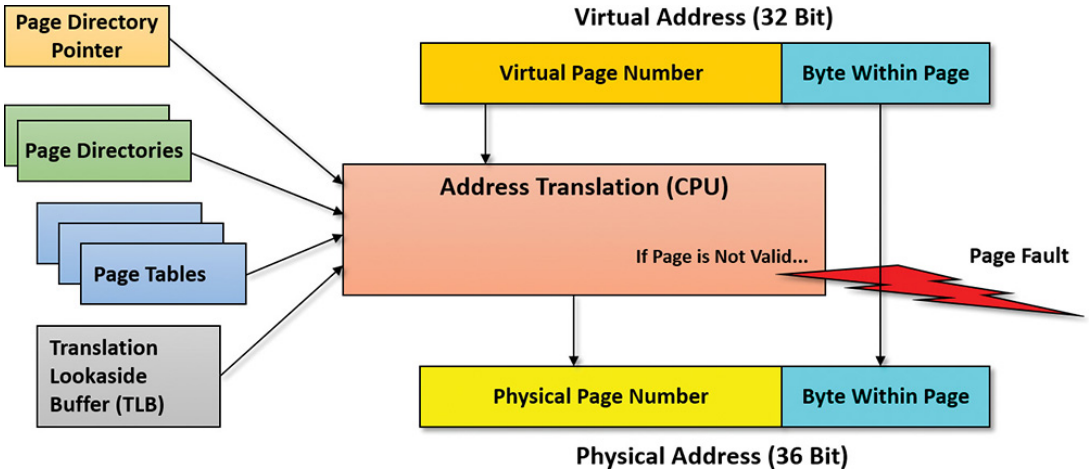


FIGURE 5-16 Virtual address translation overview.

As shown in **Figure 5-16**, the input to the translation system consists of a 32-bit virtual address (since this is the addressable range with 32 bit) and a bunch of memory-related structures (page tables, page directories, a single page directory pointer table [PDPT], and translation lookaside buffers, all described shortly). The output should be a 36-bit physical address in RAM where the actual byte is located. The number 36 comes from the way the page tables are structured and, as mentioned, dictated by the processor. When mapping small pages (the common case, shown in **Figure 5-16**), the least significant 12 bits from the virtual address are copied directly to the resulting physical address. 12 bits is exactly 4 KB—the size of a small page.

If the address cannot be translated successfully (for example, the page may not be in physical memory but resides in a page file), the CPU throws an exception known as a *page fault* that indicates to the OS that the page cannot be located. Because the CPU has no idea where to find the page (page file, mapped file, or something else), it relies on the OS to get the page from wherever it's located (if possible), fix the page tables to point to it, and request that the CPU tries translation again. (Page faults are described in the section "**Page files**" later in this chapter.)

Figure 5-17 depicts the entire process of translating x86 virtual to physical addresses.

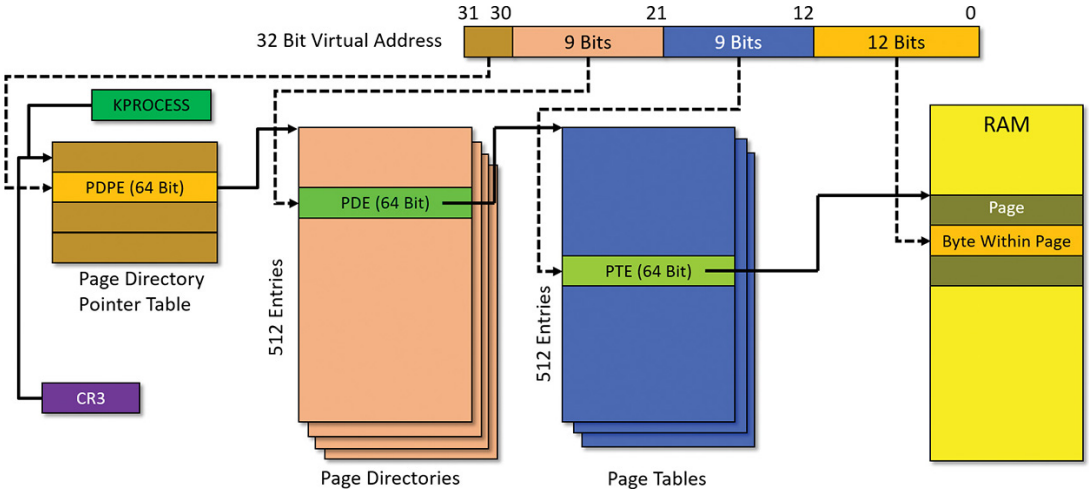


FIGURE 5-17 x86 virtual address translation.

The 32-bit virtual address to be translated is logically segregated into four parts. As you've seen, the lower 12 bits are used as-is to select a specific byte within a page. The translation process starts with a single PDPT per process, which resides in physical memory at all times. (Otherwise, how would the system locate it?) Its physical address is stored in the `KPROCESS` structure for each process. The special x86 register CR3 stores this value for the currently executing process (that is, one of its threads made the access to the virtual address). This means that when a context switch occurs on a CPU, if the new thread is running under a different process than the old thread, then the CR3 register must be loaded with the new process's page directory pointer address from its `KROCESS` structure. The PDPT must be aligned on a 32-byte boundary and must furthermore reside in the first 4 GB of RAM (because CR3 on x86 is still a 32-bit register).

Given the layout in **Figure 5-17**, the sequence of translating a virtual address to a physical one goes as follows:

1. The two most significant bits of the virtual address (bits 30 and 31) provide an index into the PDPT. This table has four entries. The entry selected—the page directory pointer entry (PDPE)—points to the physical address of a page directory.

2. A page directory contains 512 entries, one of which is selected by bits 21 to 29 (9 bits) from the virtual address. The selected page directory entry (PDE) points to the physical address of a page table.

3. A page table also contains 512 entries, one of which is selected by bits 13 to 28 (9 bits) from the virtual address. The selected page table entry (PTE) points to the physical address of the start of the page.

4. The virtual address offset (lower 12 bits) is added to the PTE pointed-to address to give the final physical address requested by the caller.

Every entry value in the various tables is also called a page frame number (PFN) because it points to a page-aligned address. Each entry is 64 bits wide—so the size of a page directory or page table is no larger than a 4 KB page—but only 24 bits are strictly necessary to describe a 64 GB physical range (combined with the 12 bits of the offset for an address range with a total of 36 bits). This means there are more bits than needed for the actual PFN values.

One of the extra bits in particular is paramount to the whole mechanism: the valid bit. This bit indicates whether the PFN data is indeed valid and therefore whether the CPU should execute the procedure just outlined. If the bit is clear, however, it indicates a page fault. The CPU raises an exception and expects the OS to handle the page fault in some meaningful way. For example, if the page in question was previously written to disk, then the memory manager should read it back to a free page in RAM, fix the PTE, and tell the CPU to try again.

Because Windows provides a private address space for each process, each process has its own PDPT, page directories, and page tables to map that process's private address space. However, the page directories and page tables that describe system space are shared among all processes (and session space is shared only among processes in a session). To avoid having multiple page tables describing the same virtual memory, the page directory entries that describe system space are initialized to point to the existing system page tables when a process is created. If the process is part of a session, session space page tables are also shared by pointing the session space page directory entries to the existing session page tables.

Page tables and page table entries

Each PDE points to a page table. A page table is a simple array of PTEs. So, too, is a PDPT. Every page table has 512 entries and each PTE maps a single page (4 KB). This means a page table can map a 2 MB address space (512 x 4 KB). Similarly, a page directory has 512 entries, each pointing to a page table. This means a page directory can map 512 x 2 MB or 1 GB of address space. This makes sense because there are four PDPEs, which together can map the entire 32-bit 4 GB address space.

For large pages, the PDE points with 11 bits to the start of a large page in physical memory, where the byte offset is taken for the low 21 bits of the original virtual address. This means such a PDE mapping a large page does not point to any page table.

The layout of a page directory and page table is essentially the same. You can use the kernel debugger `!pte` command to examine PTEs. (See the upcoming experiment "**Translating addresses**.") We'll discuss valid PTEs here and invalid PTEs in the section "**Page fault handling**." Valid PTEs have two main fields: the PFN of the physical page containing the data or of the physical address of a page in memory, and some flags that describe the state and protection of the page. (See **Figure 5-18**.)

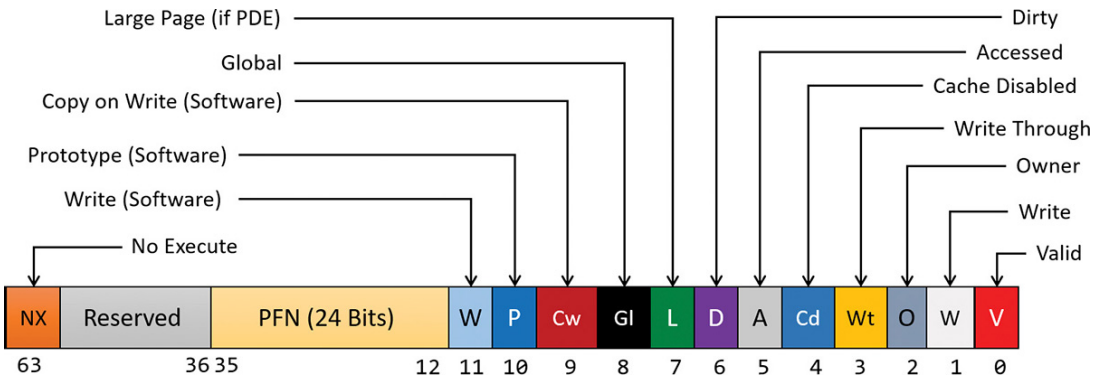


FIGURE 5-18 Valid x86 hardware PTEs.

The bits labeled "Software" and "Reserved" in **Figure 5-18** are ignored by the memory management unit (MMU) inside the CPU regardless of whether the PTE is valid. These bits are stored and interpreted by the memory manager. **Table 5-10** briefly describes the hardware-defined bits in a valid PTE.

Name of Bit	Meaning
Accessed	The page has been accessed.
Cache disabled	This disables CPU caching for that page.
Copy-on-write	The page is using copy-on-write (described earlier).
Dirty	The page has been written to.
Global	Translation applies to all processes. For example, a translation buffer flush won't affect this PTE.
Large page	This indicates that the PDE maps a 2 MB page. (Refer to the section "Large and small pages" earlier in this chapter.)
No execute	This indicates that code cannot execute in the page. (It can be used for data only.)
Owner	This indicates whether user-mode code can access the page or whether the page is limited to kernel-mode access.
Prototype	The PTE is a prototype PTE, which is used as a template to describe shared memory associated with section objects.
Valid	This indicates whether the translation maps to a page in physical memory.
Write through	This marks the page as write-through or, if the processor supports the page attribute table, write-combined. This is typically used to map video frame buffer memory.
Write	This indicates to the MMU whether the page is writable.

TABLE 5-10 PTE status and protection bits

On x86 systems, a hardware PTE contains two bits that can be changed by the MMU: the dirty bit and the accessed bit. The MMU sets the accessed bit whenever the page is read or written (provided it is not already set). The MMU sets the dirty bit whenever a write operation occurs to the page. The operating system is responsible for clearing these bits at the appropriate times. They are never cleared by the MMU.

The x86 MMU uses a write bit to provide page protection. When this bit is clear, the page is read-only. When it is set, the page is read/write. If a thread attempts to write to a page with the write bit clear, a memory-management exception occurs. In addition, the memory manager's access fault handler (described later in the section "**Page fault handling**") must determine whether the thread can be allowed to write to the page (for example, if the page was really marked copy-on-write) or whether an access violation should be generated.

Hardware versus software write bits in page table entries

The additional write bit implemented in software (refer to **Table 5-10**) is used to force an update of the dirty bit to be synchronized with updates to Windows memory management data. In a simple implementation, the memory manager would set the hardware write bit (bit 1) for any writable page. A write to any such page will cause the MMU to set the dirty bit in the PTE. Later, the dirty bit will tell the memory manager that the contents of that physical page must be written to backing store before the physical page can be used for something else.

In practice, on multiprocessor systems, this can lead to race conditions that are expensive to resolve. At any time, the MMUs of the various processors can set the dirty bit of any PTE that has its hardware write bit set. The memory manager must, at various times, update the process working set list to reflect the state of the dirty bit in a PTE. The memory manager uses a pushlock to synchronize access to the working set list. But on a multiprocessor system, even while one processor is holding the lock, the dirty bit might be changed by MMUs of other CPUs. This raises the possibility of missing an update to a dirty bit.

To avoid this, the Windows memory manager initializes both read-only and writable pages with the hardware write bit (bit 1) of their PTEs set to 0 and records the true writable state of the page in the software write bit (bit 11). On the first write access to such a page, the processor will raise a memory-management exception because the hardware write bit is clear, just as it would be for a true read-only page. In this case, though, the memory manager learns that the page actually *is* writable (via the software write bit), acquires the working set pushlock, sets the dirty bit and the hardware write bit in the PTE, updates the working set list to note that the page has been changed, releases the working set pushlock, and dismisses the exception. The hardware write operation then proceeds as usual, but the setting of the dirty bit is made to happen with the working set list pushlock held.

On subsequent writes to the page, no exceptions occur because the hardware write bit is set. The MMU will redundantly set the dirty bit,

but this is benign because the "written-to" state of the page is already recorded in the working set list. Forcing the first write to a page to go through this exception handling may seem to be excessive overhead. However, it happens only once per writable page as long as the page remains valid. Furthermore, the first access to almost any page already goes through memory-management exception handling because pages are usually initialized in the invalid state (the PTE bit 0 is clear). If the first access to a page is also the first write access to the page, the dirty bit handling just described will occur within the handling of the first-access page fault, so the additional overhead is small. Finally, on both uniprocessor and multiprocessor systems, this implementation allows flushing of the translation look-aside buffer (described in the next section) without holding a lock for each page being flushed.

Translation look-aside buffer

As you've learned, each hardware address translation requires three lookups:

- One to find the right entry in the PDPT
- One to find the right entry in the page directory (which provides the location of the page table)
- One to find the right entry in the page table

Because doing three additional memory lookups for every reference to a virtual address would quadruple the required bandwidth to memory, resulting in poor performance, all CPUs cache address translations so that repeated accesses of the same addresses don't have to be repeatedly translated. This cache is an array of associative memory called the *translation lookaside buffer* (*TLB*). Associative memory is a vector whose cells can be read simultaneously and compared to a target value. In the case of the TLB, the vector contains the virtual-to-physical page mappings of the most recently used pages, as shown in **Figure 5-19**, and the type of page protection, size, attributes, and so on applied to each page. Each entry in the TLB is like a cache entry whose tag holds por-

tions of the virtual address and whose data portion holds a physical page number, protection field, valid bit, and usually a dirty bit indicating the condition of the page to which the cached PTE corresponds. If a PTE's global bit is set (as is done by Windows for system space pages that are visible to all processes), the TLB entry isn't invalidated on process context switches.

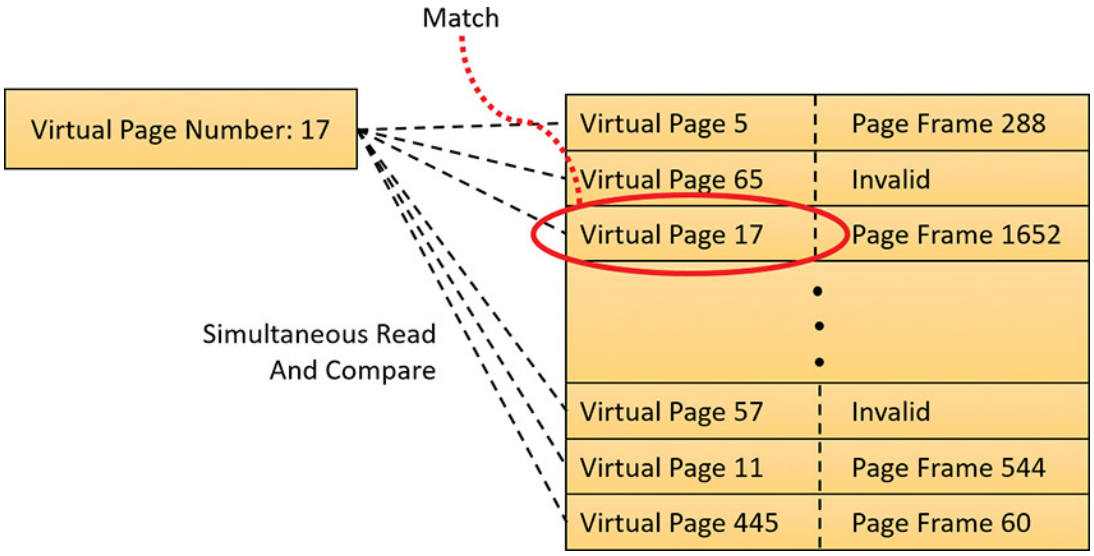


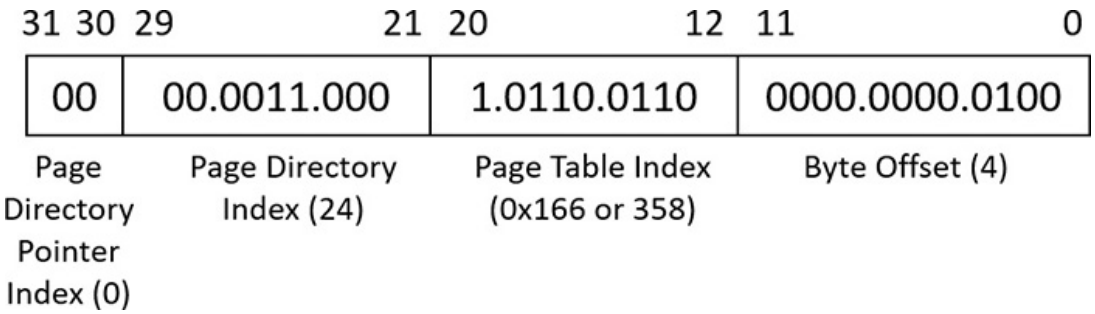
FIGURE 5-19 Accessing the TLB.

Frequently used virtual addresses are likely to have entries in the TLB, which provides extremely fast virtual-to-physical address translation and, therefore, fast memory access. If a virtual address isn't in the TLB, it might still be in memory, but multiple memory accesses are needed to find it, which makes the access time slightly slower. If a virtual page has been paged out of memory or if the memory manager changes the PTE, the memory manager is required to explicitly invalidate the TLB entry. If a process accesses it again, a page fault occurs, and the memory manager brings the page back into memory (if needed) and re-creates its PTE (which then results in an entry for it in the TLB).

EXPERIMENT: TRANSLATING ADDRESSES

To clarify how address translation works, this experiment shows an example of translating a virtual address on an x86 PAE system using the available tools in the kernel debugger to examine the PDPT, page directories, page tables, and PTEs. In this example, you'll work with a process that has virtual address 0x3166004, currently mapped to a valid physical address. In later examples, you'll see how to follow address translation for invalid addresses with the kernel debugger.

First convert 0x3166004 to binary and break it into the three fields used to translate an address. In binary, 0x3166004 is 11.0001.0110.0110.0000.0000.0100. Breaking it into the component fields yields the following:



To start the translation process, the CPU needs the physical address of the process's PDPT. This is found in the CR3 register while a thread in that process is running. You can display this address by looking at the `DirBase` field in the output of the `!process` command, as shown here:

Click here to view code image

```
lkd> !process -1 0
PROCESS 99aa3040  SessionId: 2  Cid: 1690    Peb: 03159000  ParentCid:
0920
    DirBase: 01024800  ObjectTable: b3b386c0  HandleCount: <Data Not
Accessible>
    Image: windbg.exe
```

The `DirBase` field shows that the PDPT is at physical address 0x1024800. As shown in the preceding illustration, the PDPT index field in the sample virtual address is 0. Therefore, the PDPT entry that con-

tains the physical address of the relevant page directory is the first entry in the PDPT, at physical address 0x1024800.

The kernel debugger `!pte` command displays the PDE and PTE that describe a virtual address, as shown here:

Click here to view code image

```
lkd> !pte 3166004
                    VA 03166004
PDE at C06000C0            PTE at C0018B30
contains 0000000056238867  contains 800000005DE61867
pfn 56238     ---DA--UWEV  pfn 5de61     ---DA--UW-V
```

The debugger does not show the PDPT, but it is easy to display given its physical address:

Click here to view code image

```
lkd> !dq 01024800 L4
# 1024800 00000000'53c88801 00000000'53c89801
# 1024810 00000000'53c8a801 00000000'53c8d801
```

Here we have used the debugger extension command `!dq`. This is similar to the `dq` command (display as quadwords—64 bit values) but lets us examine memory by physical rather than virtual address. Because we know the PDPT is only four entries long, we added the `L 4` length argument to keep the output uncluttered.

As illustrated, the PDPT index (the two most significant bits) from the sample virtual address equal 0, so the PDPT entry you want is the first displayed quadword. PDPT entries have a format similar to PDEs and PTEs, so you can see that this one contains a PFN of 0x53c88 (always page-aligned), for a physical address of 0x53c88000. That's the physical address of the page directory.

The `!pte` output shows the PDE address 0xC06000C0 as a virtual address, not physical. On x86 systems, the first process page directory

starts at virtual address 0xC0600000. In this case, the PDE address is 0xC0—that is, 8 bytes (the size of an entry) times 24 added to the page directory start address. Therefore, the page directory index field of the sample virtual address is 24. That means you're looking at the 25th PDE in the page directory.

The PDE provides the PFN of the needed page table. In this example, the PFN is 0x56238, so the page table starts at physical address 0x56238000. To this the MMU will add the page table index field (0x166) from the virtual address multiplied by 8 (the size of a PTE in bytes). The resulting physical address of the PTE is 0x56238B30.

The debugger shows that this PTE is at *virtual* address 0xC0018B30. Notice that the byte offset portion (0xB30) is the same as that from the physical address, as is always the case in address translation. Because the memory manager maps page tables starting at 0xC0000000, adding 0xB30 to 0xC0018000 (the 0x18 is entry 24 as you've seen) yields the virtual address shown in the kernel debugger output: 0xC0018B30. The debugger shows that the PFN field of the PTE is 0x5DE61.

Finally, you can consider the byte offset from the original address. As described, the MMU concatenates the byte offset to the PFN from the PTE, giving a physical address of 0x5DE61004. This is the physical address that corresponds to the original virtual address of 0x3166004...at the moment.

The flags bits from the PTE are interpreted to the right of the PFN number. For example, the PTE that describes the page being referenced has flags of `---DA--UW-V`. Here, `A` stands for accessed (the page has been read), `U` for user-mode accessible (as opposed to kernel-mode accessible only), `W` for writable page (rather than just readable), and `V` for valid (the PTE represents a valid page in physical memory).

To confirm the calculation of the physical address, look at the memory in question via both its virtual and its physical addresses. First, using the debugger's `dd` (display dwords) command on the virtual address, you see the following:

Click here to view code image

```
lkd> dd 3166004 L 10
03166004  00000034 00000006 00003020 0000004e
03166014  00000000 00020020 0000a000 00000014
```

And with the `!dd` command on the physical address just computed, you see the same contents:

Click here to view code image

```
lkd> !dd 5DE61004 L 10
#5DE61004 00000034 00000006 00003020 0000004e
#5DE61014 00000000 00020020 0000a000 00000014
```

You could similarly compare the displays from the virtual and physical addresses of the PTE and PDE.

---

x64 virtual address translation

Address translation on x64 is similar to x86, but with a fourth level added. Each process has a top-level extended page directory called the *page map level 4 table* that contains the physical locations of 512 third-level structures, called *page directory pointers*. The page parent directory is analogous to the x86 PAE PDPT, but there are 512 of them instead of just one, and each page parent directory is an entire page containing 512 entries instead of just four. Like the PDPT, the page parent directory's entries contain the physical locations of second-level page directories, each of which in turn contains 512 entries providing the locations of the individual page tables. Finally, the page tables, each of which contains 512 page table entries, contain the physical locations of the pages in memory. All the "physical locations" in the preceding description are stored in these structures as PFNs.

Current implementations of the x64 architecture limit virtual addresses to 48 bits. The components that make up this 48-bit virtual address and

the connection between the components for translation purposes are shown in **Figure 5-20**, and the format of an x64 hardware PTE is shown in **Figure 5-21**.

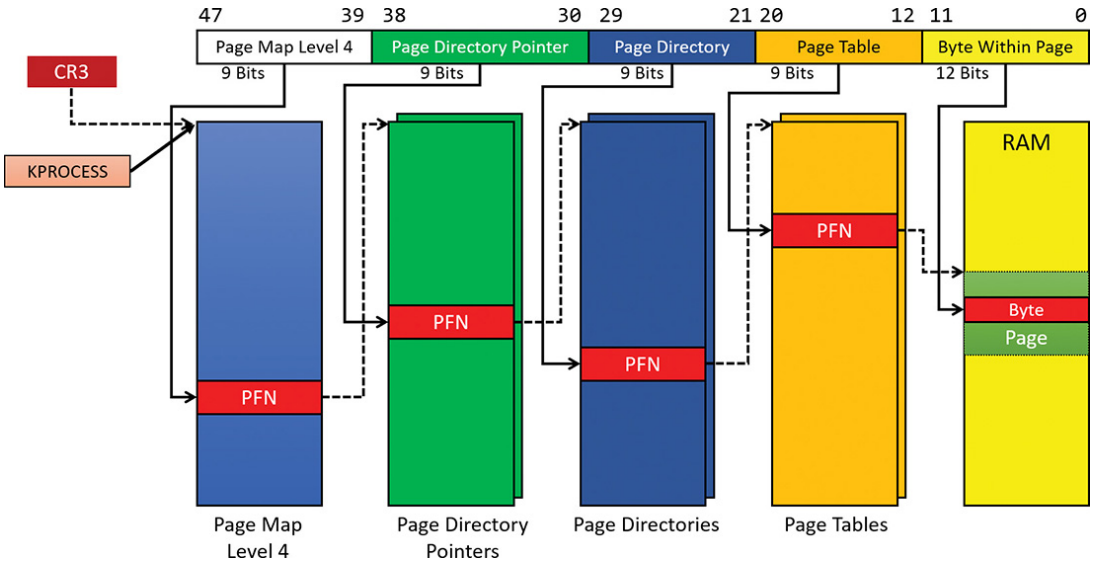


FIGURE 5-20 x64 address translation.

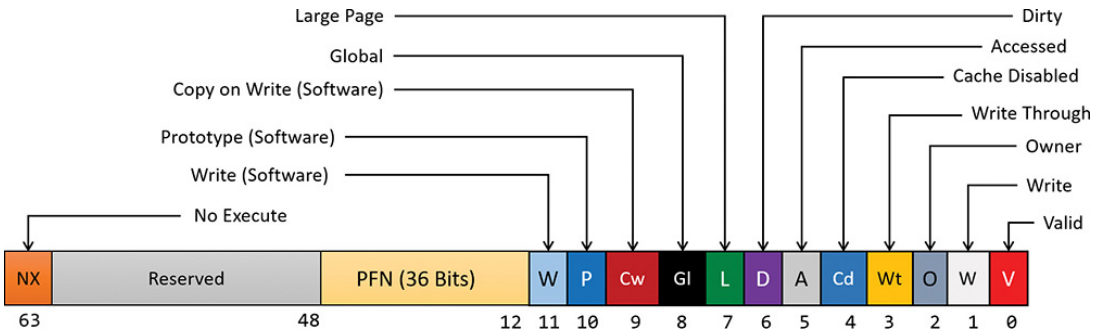


FIGURE 5-21 x64 hardware PTE.

ARM virtual address translation

Virtual address translation on ARM 32-bit processors uses a single page directory with 1,024 entries, each 32 bits in size. The translation structures are shown in **Figure 5-22**.

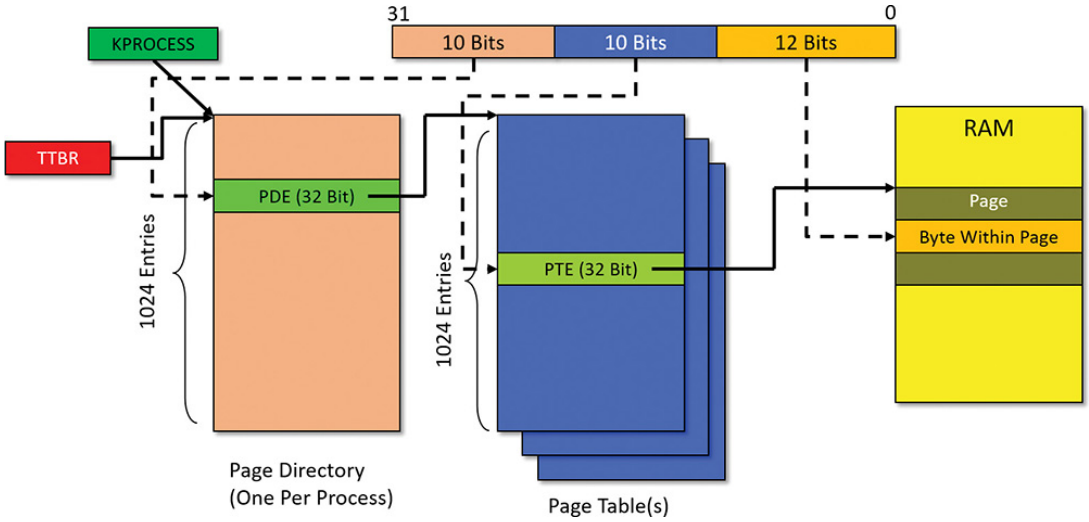


FIGURE 5-22 ARM virtual address translation structures.

Every process has a single page directory, with its physical address stored in the TTBR register (similar to the CR3 x86/x64 register). The 10 most significant bits of the virtual address select a PDE that may point to one of 1,024 page tables. A specific PTE is selected by the next 10 bits of the virtual address. Each valid PTE points to the start of a page in physical memory, where the offset is given by the lower 12 bits of the address (just as in the x86 and x64 cases). The scheme in **Figure 5-22** suggests that the addressable physical memory is 4 GB because each PTE is smaller (32 bits) than the x86/x64 case (64 bits), and indeed only 20 bits are used for the PFN. ARM processors do support a PAE mode (similar to x86), but Windows does not use this functionality. Future Windows versions may support the ARM 64-bit architecture, which will alleviate the physical address limitations as well as dramatically increase the virtual address space for processes and the system.

Curiously, the layout of valid PTE, PDE, and large page PDE are not the same. **Figure 5-23** shows the layout of a valid PTE for ARMv7, currently used by Windows. For more information, consult the official ARM documentation.

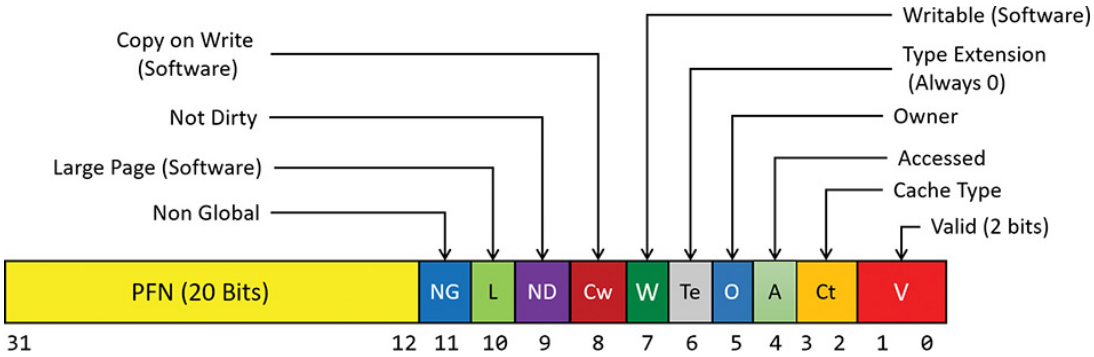


FIGURE 5-23 ARM valid PTE layout.

Page fault handling

Earlier, you saw how address translations are resolved when the PTE is valid. When the PTE valid bit is clear, this indicates that the desired page is for some reason not currently accessible to the process. This section describes the types of invalid PTEs and how references to them are resolved.

NOTE

Only the 32-bit x86 PTE formats are detailed in this section. PTEs for 64-bit and ARM systems contain similar information, but their detailed layout is not presented.

A reference to an invalid page is called a *page fault*. The kernel trap handler (see the section "Trap dispatching" in Chapter 8 in Part 2) dispatches this kind of fault to the memory manager fault handler function, `MmAccessFault`, to resolve. This routine runs in the context of the thread that incurred the fault and is responsible for attempting to resolve the fault (if possible) or raise an appropriate exception. These faults can be caused by a variety of conditions, as listed in **Table 5-11**.

TABLE 5-11 Reasons for access faults

The following section describes the four basic kinds of invalid PTEs that are processed by the access fault handler. Following that is an explanation of a special case of invalid PTEs, called *prototype PTEs,* which are used to implement shareable pages.

Invalid PTEs

If the valid bit of a PTE encountered during address translation is zero, the PTE represents an invalid page—one that will raise a memory-management exception, or page fault, upon reference. The MMU ignores the remaining bits of the PTE, so the operating system can use these bits to store information about the page that will assist in resolving the page fault.

The following list details the four kinds of invalid PTEs and their structure. These are often referred to as *software PTEs* because they are interpreted by the memory manager rather than the MMU. Some of the flags are the same as those for a hardware PTE, as described in **Table 5-10**, and some of the bit fields have either the same or similar meanings to corresponding fields in the hardware PTE.

■ **Page file** The desired page resides within a paging file. As illustrated in **Figure 5-24**, 4 bits in the PTE indicate in which of 16 possible page files the page resides, and 32 bits provide the page number within the file. The pager initiates an in-page operation to bring the page into memory and make it valid. The page file offset is always non-zero and never all ones (that is, the very first and last pages in the page file are not used for paging) to allow for other formats, described next.

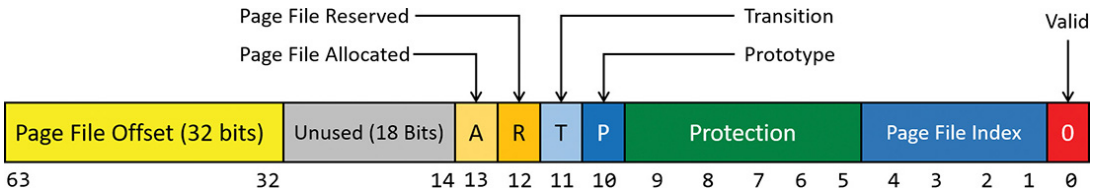


FIGURE 5-24 A PTE representing a page in a page file.

■ **Demand zero** This PTE format is the same as the page file PTE shown in the previous entry but the page file offset is zero. The desired page must be satisfied with a page of zeroes. The pager looks at the zero page list. If the list is empty, the pager takes a page from the free list and zeroes it. If the free list is also empty, it takes a page from one of the standby lists and zeroes it.

■ **Virtual Address Descriptor** This PTE format is the same as the page file PTE shown previously but in this case the page file offset field is all one. This indicates a page whose definition and backing store, if any, can be found in the process's Virtual Address Descriptor (VAD) tree. This format is used for pages that are backed by sections in mapped files. The pager finds the VAD that defines the virtual address range encompassing the virtual page and initiates an in-page operation from the mapped file referenced by the VAD. (VADs are described in more detail in the section "**Virtual address descriptors**" later in this chapter.)

■ **Transition** The transition bit is one. The desired page is in memory on either the standby, modified, or modified-no-write list or not on any list. The pager will remove the page from the list (if it is on one) and add it to the process working set. This is known as a *soft page fault* because no I/O is involved.

■ **Unknown** The PTE is zero or the page table doesn't yet exist. (The PDE that would provide the physical address of the page table contains zero.) In both cases, the memory manager must examine the VADs to determine whether this virtual address has been committed. If so, page tables are built to represent the newly committed address space. If not —that is, if the page is reserved or hasn't been defined at all—the page fault is reported as an access violation exception.

Prototype PTEs

If a page can be shared between two processes, the memory manager uses a software structure called *prototype page table entries* (*prototype PTEs*) to map these potentially shared pages. For page-file–backed sections, an array of prototype PTEs is created when a section object is first created. For mapped files, portions of the array are created on demand as each view is mapped. These prototype PTEs are part of the segment structure (described in the section "**Section objects**" later in this chapter).

When a process first references a page mapped to a view of a section object (recall that VADs are created only when the view is mapped), the memory manager uses the information in the prototype PTE to fill in the real PTE used for address translation in the process page table. When a shared page is made valid, both the process PTE and the prototype PTE point to the physical page containing the data. To track the number of process PTEs that reference a valid shared page, a counter in its PFN database entry is incremented. Thus, the memory manager can determine when a shared page is no longer referenced by any page table and thus can be made invalid and moved to a transition list or written out to disk.

When a shareable page is invalidated, the PTE in the process page table is filled in with a special PTE that points to the prototype PTE that describes the page, as shown in **Figure 5-25**. Thus, when the page is accessed, the memory manager can locate the prototype PTE using the information encoded in this PTE, which in turn describes the page being referenced.

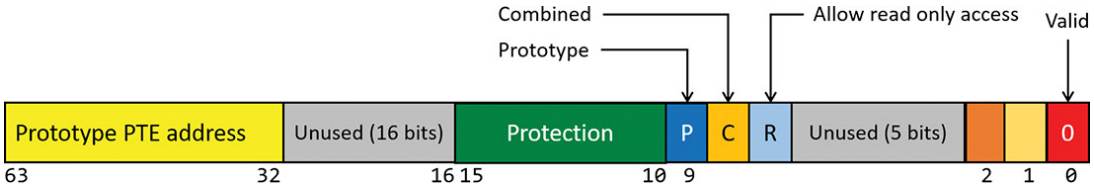


FIGURE 5-25 Structure of an invalid PTE that points to the prototype PTE.

A shared page can be in one of six different states, as described by the prototype PTE:

- **Active/valid** The page is in physical memory because of another process that accessed it.

- **Transition** The desired page is in memory on the standby or modified list (or not on any list).

- **Modified-no-write** The desired page is in memory and on the modified-no-write list. (Refer to **Table 5-11**.)

- **Demand zero** The desired page should be satisfied with a page of zeroes.

- **Page file** The desired page resides within a page file.

- **Mapped file** The desired page resides within a mapped file.

Although the format of these prototype PTEs is the same as that of the real PTEs described earlier, the prototype PTEs aren't used for address translation. They are a layer between the page table and the PFN database and never appear directly in page tables.

By having all the accessors of a potentially shared page point to a prototype PTE to resolve faults, the memory manager can manage shared pages without needing to update the page tables of each process sharing the page. For example, a shared code or data page might be paged out to disk at some point. When the memory manager retrieves the page from disk, it needs only to update the prototype PTE to point to the page's new physical location. The PTEs in each of the processes sharing

the page remain the same, with the valid bit clear and still pointing to the prototype PTE. Later, as processes reference the page, the real PTE will get updated.

Figure 5-26 illustrates two virtual pages in a mapped view. One is valid and the other is invalid. As shown, the first page is valid and is pointed to by the process PTE and the prototype PTE. The second page is in the paging file—the prototype PTE contains its exact location. The process PTE (and any other processes with that page mapped) points to this prototype PTE.

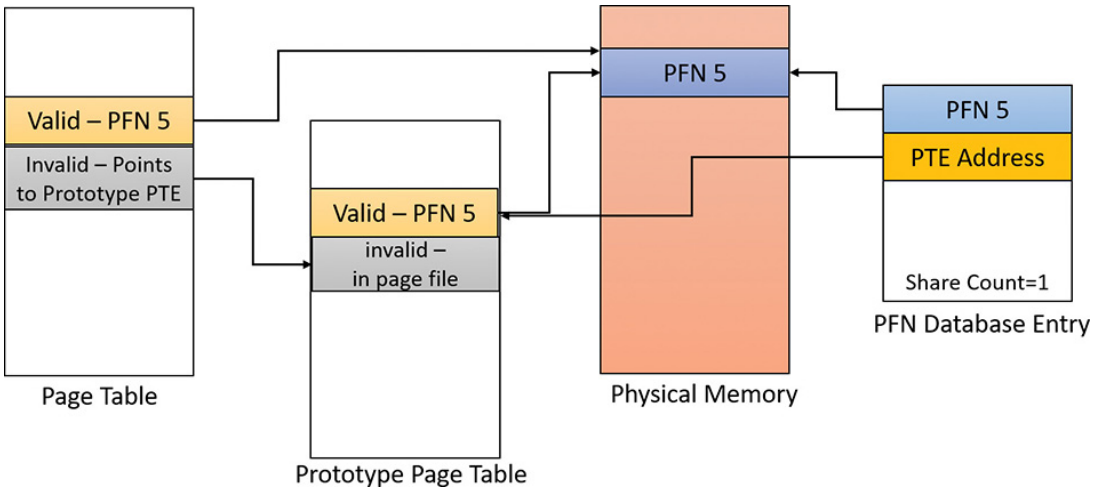


FIGURE 5-26 Prototype page table entries.

In-paging I/O

In-paging I/O occurs when a read operation must be issued to a file (paging or mapped) to satisfy a page fault. Also, because page tables are themselves pageable, the processing of a page fault can incur additional I/O if necessary when the system is loading the page table page that contains the PTE or the prototype PTE that describes the original page being referenced.

The in-page I/O operation is synchronous—that is, the thread waits on an event until the I/O completes—and isn't interruptible by asynchronous procedure call (APC) delivery. The pager uses a special modifier in the I/O request function to indicate paging I/O. Upon completion of paging I/O, the I/O system triggers an event, which wakes up the pager and allows it to continue in-page processing.

While the paging I/O operation is in progress, the faulting thread doesn't own any critical memory management synchronization objects. Other threads within the process can issue virtual memory functions and handle page faults while the paging I/O takes place. But there are a few interesting conditions that the pager must recognize when the I/O completes are exposed:

■ Another thread in the same process or a different process could have faulted the same page (called a *collided page fault* and described in the next section).

■ The page could have been deleted and remapped from the virtual address space.

■ The protection on the page could have changed.

■ The fault could have been for a prototype PTE, and the page that maps the prototype PTE could be out of the working set.

The pager handles these conditions by saving enough state on the thread's kernel stack before the paging I/O request such that when the request is complete, it can detect these conditions and, if necessary, dismiss the page fault without making the page valid. When and if the faulting instruction is reissued, the pager is again invoked and the PTE is reevaluated in its new state.

Collided page faults

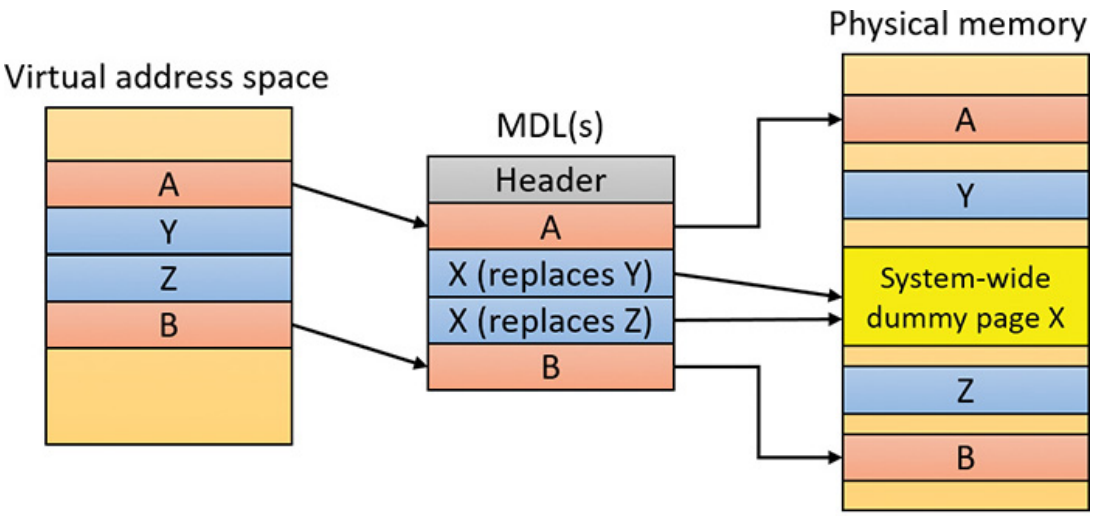
The case when another thread in the same process or a different process faults a page that is currently being in-paged is known as a *collided page fault*. The pager detects and handles collided page faults optimally because they are common occurrences in multithreaded systems. If another thread or process faults the same page, the pager detects the collided page fault, noticing that the page is in transition and that a read is in progress. (This information is in the PFN database entry.) In this case, the pager may issue a wait operation on the event specified in the PFN database entry. Alternatively, it can choose to issue a parallel I/O to protect the file systems from deadlocks. (The first I/O to complete "wins," and the others are discarded.) This event was initialized by the thread that first issued the I/O needed to resolve the fault.

When the I/O operation completes, all threads waiting on the event have their wait satisfied. The first thread to acquire the PFN database lock is responsible for performing the in-page completion operations. These operations consist of checking I/O status to ensure that the I/O operation completed successfully, clearing the read-in-progress bit in the PFN database, and updating the PTE.

When subsequent threads acquire the PFN database lock to complete the collided page fault, the pager recognizes that the initial updating has been performed because the read-in-progress bit is clear and checks the in-page error flag in the PFN database element to ensure that the in-page I/O completed successfully. If the in-page error flag is set, the PTE isn't updated, and an in-page error exception is raised in the faulting thread.

Clustered page faults

The memory manager prefetches large clusters of pages to satisfy page faults and populate the system cache. The prefetch operations read data directly into the system's page cache instead of into a working set in virtual memory. Therefore, the prefetched data does not consume virtual address space, and the size of the fetch operation is not limited to the amount of virtual address space that is available. Also, no expensive TLB-flushing inter-processor interrupt (IPI) is needed if the page will be repurposed. The prefetched pages are put on the standby list and marked as in transition in the PTE. If a prefetched page is subsequently referenced, the memory manager adds it to the working set. However, if it is never referenced, no system resources are required to release it. If any pages in the prefetched cluster are already in memory, the memory manager does not read them again. Instead, it uses a dummy page to represent them so that an efficient single large I/O can still be issued, as shown in **Figure 5-27**.



Pages Y and Z are already in memory, so the corresponding MDL entries point to the system-wide dummy page

FIGURE 5-27 Usage of dummy page during virtual-address-to-physical-address mapping in an MDL.

In the figure, the file offsets and virtual addresses that correspond to pages A, Y, Z, and B are logically contiguous, although the physical pages themselves are not necessarily contiguous. Pages A and B are nonresident, so the memory manager must read them. Pages Y and Z are already resident in memory, so it is not necessary to read them. (In

fact, they might already have been modified since they were last read in from their backing store, in which case it would be a serious error to overwrite their contents.) However, reading pages A and B in a single operation is more efficient than performing one read for page A and a second read for page B. Therefore, the memory manager issues a single read request that comprises all four pages (A, Y, Z, and B) from the backing store. Such a read request includes as many pages as it makes sense to read, based on the amount of available memory, the current system usage, and so on.

When the memory manager builds the MDL that describes the request, it supplies valid pointers to pages A and B. However, the entries for pages Y and Z point to a single system-wide dummy page X. The memory manager can fill the dummy page X with the potentially stale data from the backing store because it does not make X visible. However, if a component accesses the Y and Z offsets in the MDL, it sees the dummy page X instead of Y and Z.

The memory manager can represent any number of discarded pages as a single dummy page, and that page can be embedded multiple times in the same MDL or even in multiple concurrent MDLs that are being used for different drivers. Consequently, the contents of the locations that represent the discarded pages can change at any time. (See **Chapter 6** for more on MDLs.)

Page files

Page files store modified pages that are still in use by some process but have had to be written to disk because they were unmapped or memory pressure resulted in a trim. Page file space is reserved when the pages are initially committed, but the actual optimally clustered page file locations cannot be chosen until pages are written out to disk.

When the system boots, the Session Manager process (Smss.exe) reads the list of page files to open by examining the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PagingFiles registry value. This multistring registry value

contains the name, minimum size, and maximum size of each paging file. Windows supports up to 16 paging files on x86 and x64 and up to 2 page files on ARM. On x86 and x64 systems, each page file can be up to 16 TB in size, while the maximum is 4 GB on ARM systems. Once open, the page files can't be deleted while the system is running because the System process maintains an open handle to each page file.

Because the page file contains parts of process and kernel virtual memory, for security reasons, the system can be configured to clear the page file at system shutdown. To enable this, set the `ClearPageFileAtShutdown` registry value in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management key to `1`. Otherwise, after shutdown, the page file will contain whatever data happened to have been paged out while the system was up. This data could then be accessed by someone who gained physical access to the machine.

If the minimum and maximum paging file sizes are both zero (or not specified), this indicates a system-managed paging file. Windows 7 and Server 2008 R2 used a simple scheme based on RAM size alone as follows:

- **Minimum size** Set to the amount of RAM or 1 GB, whichever is larger
- **Maximum size** Set to 3 * RAM or 4 GB, whichever is larger

These settings are not ideal. For example, today's laptops and desktop machines can easily have 32 GB or 64 GB of RAM, and server machines can have hundreds of gigabytes of RAM. Setting the initial page file size to the size of RAM may result in a considerable loss of disk space, especially if disk sizes are relatively small and based on solid-state device (SSD). Furthermore, the amount of RAM in a system is not necessarily indicative of the typical memory workload on that system.

The current implementation uses a more elaborate scheme to derive a "good" minimum page file size based not only on RAM size, but also on page file history usage and other factors. As part of page-file creation

and initialization, Smss.exe calculates page file minimum sizes based on four factors, stored in global variables:

- **RAM (`SmpDesiredPfSizeBasedOnRAM`)** This is the recommended page file size based on RAM.

- **Crash dump (`SmpDesiredPfSizeForCrashDump`)** This is the recommended page file size needed to be able to store a crash dump.

- **History (`SmpDesiredPfSizeBasedOnHistory`)** This is the recommended page file size based on usage history. Smss.exe uses a timer that triggers once an hour and records the page file usage.

- **Apps (`SmpDesiredPfSizeForApps`)** This is the recommended page file for Windows apps.

- These values are computed as shown in **Table 5-12**.

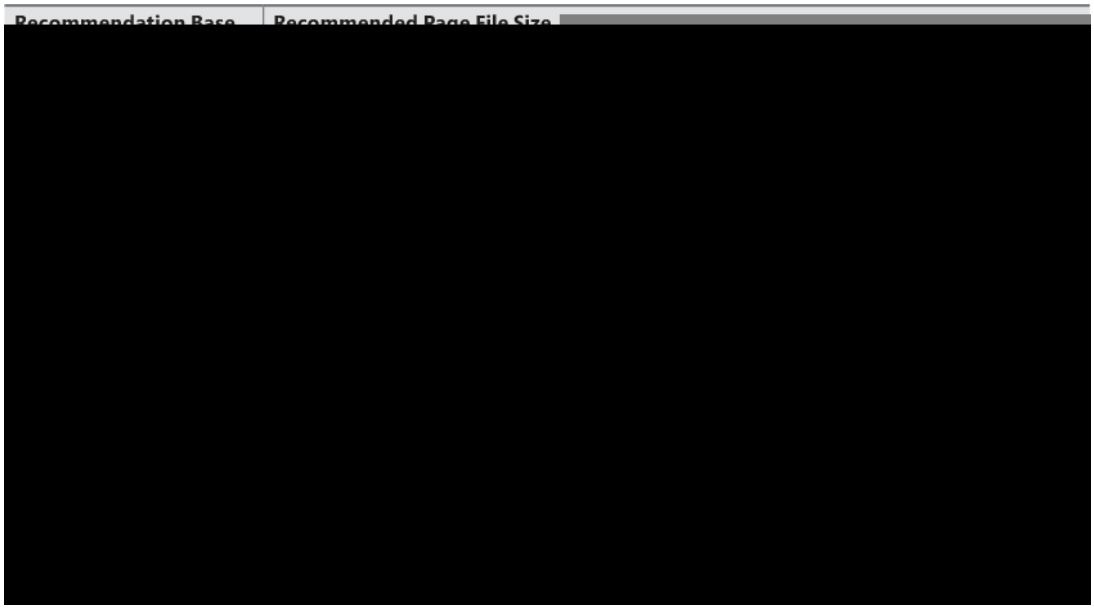


TABLE 5-12 Base calculation for page file size recommendation

The maximum page file size for a system-managed size is set at three times the size of RAM or 4 GB, whichever is larger. The minimum (initial) page file size is determined as follows:

- If it's the first system-managed page file, then the base size is set based on page file history (refer to **Table 5-12**). Otherwise, the base size

is based on RAM.

■ If it's the first system-managed page file:

• If the base size is smaller than the computed page file size for apps (`SmpDesiredPfSizeForApps`), then set the new base as the size computed for apps (refer to **Table 5-12**).

• If the (new) base size is smaller than the computed size for crash dumps (`SmpDesiredPfSizeForCrashDump`), then set the new base to be the size computed for crash dumps.

EXPERIMENT: VIEWING PAGE FILES

To view the list of page files, look in the registry at the `PagingFiles` value in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management key. This entry contains the paging file configuration settings modified through the Advanced System Settings dialog box. To access these settings, follow these steps:

1. Open Control Panel.

2. Click **System and Security** and then click **System**. This opens the System Properties dialog box, which you can also access by right-clicking on **Computer** in Explorer and selecting **Properties**.

3. Click **Advanced System Settings**.

4. In the Performance area, click **Settings**. This opens the Performance Options dialog box.

5. Click the **Advanced** tab.

6. In the Virtual Memory area, click **Change**.

EXPERIMENT: VIEWING PAGE FILE RECOMMENDED SIZES

To view the actual variables calculated in **Table 5-12**, follow these steps (this experiment was done using an x86 Windows 10 system):

1. Start local kernel debugging.

2. Locate Smss.exe processes:

Click here to view code image

```
lkd> !process 0 0 smss.exe
PROCESS 8e54bc40  SessionId: none  Cid: 0130    Peb:
02bab000  ParentCid:
0004
```

DirBase: bffe0020 ObjectTable: 8a767640 HandleCount: <Data Not Accessible>
Image: smss.exe

PROCESS 9985bc40 SessionId: 1 Cid: 01d4 Peb: 02f9c000 ParentCid: 0130
DirBase: bffe0080 ObjectTable: 00000000 HandleCount: 0.
Image: smss.exe

PROCESS a122dc40 SessionId: 2 Cid: 02a8 Peb: 02fcd000 ParentCid: 0130
DirBase: bffe0320 ObjectTable: 00000000 HandleCount: 0.
Image: smss.exe

3. Locate the first one (with a session ID of `none`), which is the master Smss.exe. (Refer to **Chapter 2** for more details.)

4. Switch the debugger context to that process:

Click here to view code image

```
lkd> .process /r /p 8e54bc40
Implicit process is now 8e54bc40
Loading User Symbols
..
```

5. Show the four variables described in the previous section. (Each one is 64 bits in size.)

Click here to view code image

```
lkd> dq smss!SmpDesiredPfSizeBasedOnRAM L1
00974cd0  00000000'4fff1a00
lkd> dq smss!SmpDesiredPfSizeBasedOnHistory L1
00974cd8  00000000'05a24700
lkd> dq smss!SmpDesiredPfSizeForCrashDump L1
00974cc8  00000000'1ffecd55
```

```
lkd> dq smss!SmpDesiredPfSizeForApps L1
00974ce0  00000000'00000000
```

6. Because there is a single volume (C:\) on this machine, a single page file would be created. Assuming it wasn't specifically configured, it would be system managed. You can look at the actual file size of C:\PageFile.Sys on disk or use the `!vm` debugger command:

Click here to view code image

```
lkd> !vm 1
Page File: \??\C:\pagefile.sys
  Current:    524288 Kb  Free Space:    524280 Kb
  Minimum:    524288 Kb  Maximum:      8324476 Kb
Page File: \??\C:\swapfile.sys
  Current:    262144 Kb  Free Space:    262136 Kb
  Minimum:    262144 Kb  Maximum:      4717900 Kb
No Name for Paging File
  Current:  11469744 Kb  Free Space:  11443108 Kb
  Minimum:  11469744 Kb  Maximum:     11469744 Kb
...
```

Notice the minimum size of C:\PageFIle.sys (524288 KB). (We'll discuss the other page file entries in the next section). According to the variables, `SmpDesiredPfSizeForCrashDump` is the largest, so must be the determining factor (0x1FFECD55 = 524211 KB), which is very close to the listed value. (Page file sizes round up to multiple of 64 MB.)

To add a new page file, Control Panel uses the internal `NtCreatePagingFile` system service defined in Ntdll.dll. (This requires the `SeCreatePagefilePrivilege`.) Page files are always created as non-compressed files, even if the directory they are in is compressed. Their names are *PageFile.Sys* (except some special ones described in the next section). They are created in the root of partitions with the Hidden file attribute so they are not immediately visible. To keep new page files

from being deleted, a handle is duplicated into the `System` process so that even after the creating process closes the handle to the new page file, a handle is nevertheless always open to it.

The swap file

In the UWP apps world, when an app goes to the background—for example, it is minimized—the threads in that process are suspended so that the process does not consume any CPU. The private physical memory used by the process can potentially be reused for other processes. If memory pressure is high, the private working set (physical memory used by the process) may be swapped out to disk to allow the physical memory to be used for other processes.

Windows 8 added another page file called a *swap file*. It is essentially the same as a normal page file but is used exclusively for UWP apps. It's created on client SKUs only if at least one normal page file was created (the normal case). Its name is *SwapFile.sys* and it resides in the system root partition—for example, C:\SwapFile.Sys.

After the normal page files are created, the HKLM\System\CurrentControlSet\Control\Session Manager\Memory Management registry key is consulted. If a DWORD value named `SwapFileControl` exists and its value is zero, swap file creation is aborted. If a value named `SwapFile` exists, it's read as a string with the same format as a normal page file, with a filename, an initial size, and a maximum size. The difference is that a value of zero for the sizes is interpreted as no swap file creation. These two registry values do not exist by default, which results in the creation of a SwapFile.sys file on the system root partition with a minimum size of 16 MB on fast (and small) disks (for example, SSD) or 256 MB on slow (or large SSD) disks. The maximum size of the swap file is set to 1.5 * RAM or 10 percent of the system root partition size, whichever is smaller. See **Chapter 7** in this book and Chapter 8, "System mechanisms," and Chapter 9, "Management mechanisms," in Part 2 for more on UWP apps.

NOTE

The swap file is not counted against the maximum page files supported.

The virtual page file

The `!vm` debugger command hints at another page file called “No Name for Paging File.” This is a virtual page file. As the name suggests, it has no actual file, but is used indirectly as the backing store for memory compression (described later in this chapter in the section “**Memory compression**”). It is large, but its size is set arbitrarily as not to run out of free space. The invalid PTEs for pages that have been compressed point to this virtual page file and allow the memory compression store to get to the compressed data when needed by interpreting the bits in the invalid PTE leading to the correct store, region, and index.

EXPERIMENT: VIEWING SWAP FILE AND VIRTUAL PAGE FILE INFORMATION

The `!vm` debugger command shows the information on all page files, including the swap file and the virtual page file:

Click here to view code image

```
lkd> !vm 1
Page File: \??\C:\pagefile.sys
  Current:    524288 Kb  Free Space:    524280 Kb
  Minimum:    524288 Kb  Maximum:      8324476 Kb
Page File: \??\C:\swapfile.sys
  Current:    262144 Kb  Free Space:    262136 Kb
  Minimum:    262144 Kb  Maximum:      4717900 Kb
No Name for Paging File
  Current:  11469744 Kb  Free Space:  11443108 Kb
  Minimum:  11469744 Kb  Maximum:     11469744 Kb
```

On this system, the swap file minimum size is 256 MB, as the system is a Windows 10 virtual machine. (The VHD behind the disk is considered a slow disk.) The maximum size of the swap file is about 4.5 GB, as the RAM on the system is 3 GB and disk partition size is 64 GB (the minimum of 4.5 GB and 6.4 GB).

Commit charge and the system commit limit

We are now in a position to more thoroughly discuss the concepts of commit charge and the system commit limit.

Whenever virtual address space is created—for example, by a `VirtualAlloc` (for committed memory) or `MapViewOfFile` call—the system must ensure that there is room to store it, either in RAM or in backing store, before successfully completing the create request. For mapped memory (other than sections mapped to the page file), the file associated with the mapping object referenced by the `MapViewOfFile` call provides the required backing store. All other virtual allocations

rely on system-managed shared resources for storage: RAM and the paging file(s). The purpose of the system commit limit and commit charge is to track all uses of these resources to ensure they are never overcommitted—that is, that there is never more virtual address space defined than there is space to store its contents, either in RAM or in backing store (on disk).

NOTE

This section makes frequent references to paging files. It is possible (though not generally recommended) to run Windows without any paging files. Essentially, this means that when RAM is exhausted, there is no room to grow and memory allocations fail, generating a blue screen. You can consider every reference to paging files here to be qualified by "if one or more paging files exist."

Conceptually, the system commit limit represents the total committed virtual address space that can be created in addition to virtual allocations that are associated with their own backing store—that is, in addition to sections mapped to files. Its numeric value is simply the amount of RAM available to Windows plus the current sizes of any page files. If a page file is expanded or new page files are created, the commit limit increases accordingly. If no page files exist, the system commit limit is simply the total amount of RAM available to Windows.

Commit charge is the system-wide total of all committed memory allocations that must be kept in either RAM or in a paging file. From the name, it should be apparent that one contributor to commit charge is process-private committed virtual address space. However, there are many other contributors, some of them not so obvious.

Windows also maintains a per-process counter called the *process page file quota*. Many of the allocations that contribute to commit charge also contribute to the process page file quota. This represents each process's private contribution to the system commit charge. Note, however, that

this does not represent current page file usage. It represents the potential or maximum page file usage, should all these allocations have to be stored there.

The following types of memory allocations contribute to the system commit charge and, in many cases, to the process page file quota. (Some of these will be described in detail in later sections of this chapter.)

- **Private committed memory** This is memory allocated with the `VirtualAlloc` call with the `MEM_COMMIT` option. This is the most common type of contributor to the commit charge. These allocations are also charged to the process page file quota.

- **Page-file-backed mapped memory** This is memory allocated with a `MapViewOfFile` call that references a section object, which in turn is not associated with a file. The system uses a portion of the page file as the backing store instead. These allocations are not charged to the process page file quota.

- **Copy-on-write regions of mapped memory (even if it is associated with ordinary mapped** files) The mapped file provides backing store for its own unmodified content. However, should a page in the copy-on-write region be modified, it can no longer use the original mapped file for backing store. It must be kept in RAM or in a paging file. These allocations are not charged to the process page file quota.

- **Non-paged and paged pool and other allocations in system space that are not backed by explicitly associated files** Even the currently free regions of the system memory pools contribute to commit charge. The non-pageable regions are counted in the commit charge even though they will never be written to the page file because they permanently reduce the amount of RAM available for private pageable data. These allocations are not charged to the process page file quota.

- **Kernel stacks** Threads' stacks when executing in kernel mode.

■ **Page tables** Most of these are themselves pageable, and they are not backed by mapped files. However, even if they are not pageable, they occupy RAM. Therefore, the space required for them contributes to commit charge.

■ **Space for page tables that are not yet actually allocated** As you'll see, where large areas of virtual space have been defined but not yet referenced (for example, private committed virtual space), the system need not actually create page tables to describe it. However, the space for these as-yet-nonexistent page tables is charged to commit charge to ensure that the page tables can be created when they are needed.

■ **Allocations of physical memory made via the Address Windowing Extension (AWE) APIs** As discussed previously, consume physical memory directly.

For many of these items, the commit charge may represent the potential use of storage rather than its actual use. For example, a page of private committed memory does not actually occupy either a physical page of RAM or the equivalent page file space until it's been referenced at least once. Until then, it is a *demand-zero page* (described later). But commit charge accounts for such pages when the virtual space is first created. This ensures that when the page is later referenced, actual physical storage space will be available for it.

A region of a file mapped as copy-on-write has a similar requirement. Until the process writes to the region, all pages in it are backed by the mapped file. However, the process may write to any of the pages in the region at any time. When that happens, those pages are thereafter treated as private to the process. Their backing store is, thereafter, the page file. Charging the system commit for them when the region is first created ensures that there will be private storage for them later, if and when the write accesses occur.

A particularly interesting case occurs when reserving private memory and later committing it. When the reserved region is created with `VirtualAlloc`, system commit charge is not charged for the actual vir-

tual region. On Windows 8 and Server 2012 and earlier versions, it is charged for any new page table pages that will be required to describe the region, even though these might not yet exist or even be eventually needed. Starting with Windows 8.1 and Server 2012 R2, page table hierarchies for reserved regions are not charged immediately; this means that huge reserved memory regions can be allocated without exhausting page tables. This becomes important in some security features, such as Control Flow Guard (CFG, see **Chapter 7** for more details). If the region or a part of it is later committed, system commit is charged to account for the size of the region (and page tables), as is the process page file quota.

To put it another way, when the system successfully completes, for example, a `VirtualAlloc` commit or `MapViewOfFile` call, it makes a commitment that the necessary storage will be available when needed, even if it wasn't needed at that moment. Thus, a later memory reference to the allocated region can never fail for lack of storage space. (Of course, it could still fail for other reasons, such as page protection, the region being deallocated, and so on.) The commit charge mechanism allows the system to keep this commitment.

The commit charge appears in the Performance Monitor counters as Memory: Committed Bytes. It is also the first of the two numbers displayed on the Task Manager's Performance tab with the legend Commit (the second being the commit limit), and it is displayed by Process Explorer's System Information Memory tab as Commit Charge – Current.

The process page file quota appears in the performance counters as Process: Page File Bytes. The same data appears in the Process: Private Bytes performance counter. (Neither term exactly describes the true meaning of the counter.)

If the commit charge ever reaches the commit limit, the memory manager will attempt to increase the commit limit by expanding one or more page files. If that is not possible, subsequent attempts to allocate virtual memory that uses commit charge will fail until some existing

committed memory is freed. The performance counters listed in **Table 5-13** allow you to examine private committed memory usage on a system-wide, per-process, or per-page-file, basis.

TABLE 5-13 Committed memory and page file performance counters

Commit charge and page file size

The counters in **Table 5-13** can assist you in choosing a custom page file size. The default policy based on the amount of RAM works acceptably for most machines, but depending on the workload, it can result in a page file that's unnecessarily large or not large enough.

To determine how much page-file space your system really needs based on the mix of applications that have run since the system booted, examine the peak commit charge in the Memory tab of Process Explorer's System Information display. This number represents the peak amount of page file space since the system booted that would have been needed if the system had to page out the majority of private committed virtual memory (which rarely happens).

If the page file on your system is too big, the system will not use it any more or less. In other words, increasing the size of the page file does not change system performance. It simply means the system can have more committed virtual memory. If the page file is too small for the mix of applications you are running, you might get a "system running low

on virtual memory" error message. In this case, check to see whether a process has a memory leak by examining the process private bytes count. If no process appears to have a leak, check the system paged pool size. If a device driver is leaking paged pool, this might also explain the error. Refer to the "**Troubleshooting a pool leak**" experiment in the "**Kernel-mode heaps (system memory pools)**" section for information on troubleshooting a pool leak.

EXPERIMENT: VIEWING PAGE FILE USAGE WITH TASK MANAGER

You can view committed memory usage with Task Manager. To do so, click its **Performance** tab. You'll see the following counters related to page files:

The system commit total is displayed under the **Committed** label as two numbers. The first number represents *potential* page file usage, not actual page file usage. It is how much page file space would be used if all the private committed virtual memory in the system had to be paged out all at once. The second number displayed is the *commit limit*, which is the maximum virtual memory usage that the system can support before running out of virtual memory. (This includes virtual memory backed in physical memory as well as by the paging files.) The commit limit is essentially the size of RAM plus the current size of the paging files. It therefore does not account for possible page file expansion.

Process Explorer's System Information display shows an additional piece of information about system commit usage—namely, the percent-

age of the peak as compared to the limit and the current usage as compared to the limit:

Stacks

Whenever a thread runs, it must have access to a temporary storage location in which to store function parameters, local variables, and the return address after a function call. This part of memory is called a *stack*. On Windows, the memory manager provides two stacks for each thread: the user stack and the kernel stack, as well as per-processor stacks called DPC stacks. **Chapter 2** briefly discussed how system calls cause the thread to switch from a user stack to its kernel stack. Now we'll look at some extra services the memory manager provides to efficiently use stack space.

User stacks

When a thread is created, the memory manager automatically reserves a predetermined amount of virtual memory, which by default is 1 MB. This amount can be configured in the call to the `CreateThread` or `CreateRemoteThread(Ex)` function or when compiling the application by using the `/STACK:reserve` switch in the Microsoft C/C++ compiler, which stores the information in the image header. Although 1 MB is reserved, only the first page of the stack will be committed (unless the PE header of the image specifies otherwise), along with a guard page. When a thread's stack grows large enough to touch the guard page, an exception occurs, causing an attempt to allocate another guard. Through this mechanism, a user stack doesn't immediately consume all 1 MB of committed memory but instead grows with demand. (However, it will never shrink back.)

EXPERIMENT: CREATING THE MAXIMUM NUMBER OF THREADS

With only 2 GB of user address space available to each 32-bit process, the relatively large memory that is reserved for each thread's stack allows for an easy calculation of the maximum number of threads that a process can support: a little less than 2,048, for a total of nearly 2 GB of memory (unless the `increaseuserva` BCD option is used and the image is large address space aware). By forcing each new thread to use the smallest possible stack reservation size, 64 KB, the limit can grow to about 30,000 threads. You can test this for yourself by using the TestLimit utility from Sysinternals. Here is some sample output:

Click here to view code image

```
C:\Tools\Sysinternals>Testlimit.exe -t -n 64

Testlimit v5.24 - test Windows limits
Copyright (C) 2012-2015 Mark Russinovich
Sysinternals - www.sysinternals.com

Process ID: 17260

Creating threads with 64 KB stacks...
Created 29900 threads. Lasterror: 8
```

If you attempt this experiment on a 64-bit Windows installation (with 128 TB of user address space available), you would expect to see potentially hundreds of thousands of threads created (assuming sufficient memory was available). Interestingly, however, TestLimit actually creates fewer threads than on a 32-bit machine. This has to do with the fact that Testlimit.exe is a 32-bit application and thus runs under the Wow64 environment. (See Chapter 8 in Part 2 for more information on Wow64.) Each thread will therefore have not only its 32-bit Wow64 stack but also its 64-bit stack, thus consuming more than twice the memory, while keeping only 2 GB of address space. To properly test the thread-creation limit on 64-bit Windows, use the Testlimit64.exe binary instead.

You will need to terminate TestLimit with Process Explorer or Task Manager. You cannot use Ctrl+C to break the application because this operation itself creates a new thread, which will not be possible once memory is exhausted.

Kernel stacks

Although user stack sizes are typically 1 MB, the amount of memory dedicated to the kernel stack is significantly smaller: 12 KB on 32-bit systems and 16 KB on 64-bit systems, followed by another guard page, for a total of 16 or 20 KB of virtual address space. Code running in the kernel is expected to have less recursion than user code, as well as contain more efficient variable use and keep stack buffer sizes low. Because kernel stacks live in system address space (which is shared by all processes), their memory usage has a bigger impact of the system.

Although kernel code is usually not recursive, interactions between graphics system calls handled by Win32k.sys and its subsequent callbacks into user mode can cause recursive re-entries in the kernel on the same kernel stack. As such, Windows provides a mechanism for dynamically expanding and shrinking the kernel stack from its initial size. As each additional graphics call is performed from the same thread, another 16 KB kernel stack is allocated. (This happens anywhere in system address space. The memory manager provides the ability to jump stacks when nearing the guard page.) Whenever each call returns to the caller (unwinding), the memory manager frees the additional kernel stack that had been allocated, as shown in **Figure 5-28**. This mechanism allows reliable support for recursive system calls as well as efficient use of system address space. It is provided for use by driver developers when performing recursive callouts through the `KeExpandKernelStackAndCallout(Ex)` APIs, as necessary.

FIGURE 5-28 Kernel stack jumping.

EXPERIMENT: VIEWING KERNEL-STACK USAGE

You can use the RamMap tool from Sysinternals to display the physical memory currently being occupied by kernel stacks. Here's a screenshot from the **Use Counts** tab:

To view kernel-stack usage, try the following:

1. Repeat the previous TestLimit experiment, but don't terminate TestLimit yet.

2. Switch to RamMap.

3. Open the **File** menu and select **Refresh** (or press **F5**). You should see a much higher kernel stack size:

Running TestLimit a few more times (without closing previous instances) would easily exhaust physical memory on a 32-bit system, and this limitation results in one of the primary limits on system-wide 32-bit thread count.

DPC stack

Windows keeps a per-processor DPC stack available for use by the system whenever DPCs are executing. This approach isolates the DPC code from the current thread's kernel stack. (This is unrelated to the DPC's actual operation because DPCs run in arbitrary thread context. See **Chapter 6** for more on DPCs.) The DPC stack is also configured as the initial stack for handling the `sysenter` (x86), `svc` (ARM), or `syscall` (x64) instruction during a system call. The CPU is responsible for switching the stack when these instructions are executed, based on one of the model-specific registers (MSRs on x86/x64). However, Windows does not want to reprogram the MSR for every context switch because

that is an expensive operation. Windows therefore configures the per-processor DPC stack pointer in the MSR.

Virtual address descriptors

The memory manager uses a demand-paging algorithm to know when to load pages into memory, waiting until a thread references an address and incurs a page fault before retrieving the page from disk. Like copy-on-write, demand paging is a form of *lazy evaluation*—waiting to perform a task until it is required.

The memory manager uses lazy evaluation not only to bring pages into memory but also to construct the page tables required to describe new pages. For example, when a thread commits a large region of virtual memory with `VirtualAlloc`, the memory manager could immediately construct the page tables required to access the entire range of allocated memory. But what if some of that range is never accessed? Creating page tables for the entire range would be a wasted effort. Instead, the memory manager waits to create a page table until a thread incurs a page fault. It then creates a page table for that page. This method significantly improves performance for processes that reserve and/or commit a lot of memory but access it sparsely.

The virtual address space that would be occupied by such as-yet-nonexistent page tables is charged to the process page file quota and to the system commit charge. This ensures that space will be available for them should they actually be created. With the lazy-evaluation algorithm, allocating even large blocks of memory is a fast operation. When a thread allocates memory, the memory manager must respond with a range of addresses for the thread to use. To do this, the memory manager maintains another set of data structures to keep track of which virtual addresses have been reserved in the process's address space and which have not. These data structures are known as *Virtual Address Descriptors* (*VADs*). VADs are allocated in non-paged pool.

Process VADs

For each process, the memory manager maintains a set of VADs that describes the status of the process's address space. VADs are organized into a self-balancing AVL tree (named after its inventors, Adelson-Velsky and Landis, where the heights of the two child subtrees of any node differ by at most 1; this makes the insertion, lookup, and deletion very fast). On average, this results in the fewest comparisons when searching for a VAD corresponding with a virtual address. There is one VAD for each virtually contiguous range of not-free virtual addresses that all have the same characteristics (reserved versus committed versus mapped, memory access protection, and so on). **Figure 5-29** shows a diagram of a VAD tree.

FIGURE 5-29 VADs.

When a process reserves address space or maps a view of a section, the memory manager creates a VAD to store any information supplied by the allocation request, such as the range of addresses being reserved, whether the range will be shared or private, whether a child process can inherit the contents of the range, and the page protection applied to pages in the range.

When a thread first accesses an address, the memory manager must create a PTE for the page containing the address. To do so, it finds the VAD whose address range contains the accessed address and uses the information it finds to fill in the PTE. If the address falls outside the range covered by the VAD or in a range of addresses that are reserved

but not committed, the memory manager knows that the thread didn't allocate the memory before attempting to use it and therefore generates an access violation.

EXPERIMENT: VIEWING VADS

You can use the kernel debugger's `!vad` command to view the VADs for a given process. First find the address of the root of the VAD tree with the `!process` command. Then specify that address to the `!vad` command, as shown in the following example of the VAD tree for a process running Explorer.exe:

Click here to view code image

```
lkd> !process 0 1 explorer.exe
PROCESS ffffc8069382e080
    SessionId: 1  Cid: 43e0    Peb: 00bc5000  ParentCid: 0338
    DirBase: 554ab7000  ObjectTable: ffffda8f62811d80  HandleCount:
823.
    Image: explorer.exe
    VadRoot ffffc806912337f0 Vads 505 Clone 0 Private 5088. Modified
2146. Locked 0.
...
lkd> !vad ffffc8068ae1e470
VAD          Level     Start      End Commit
ffffc80689bc52b0  9       640       64f      0 Mapped       READWRITE
Pagefile section, shared commit 0x10
ffffc80689be6900  8       650       651      0 Mapped       READONLY
Pagefile section, shared commit 0x2
ffffc80689bc4290  9       660       675      0 Mapped       READONLY
Pagefile section, shared commit 0x16
ffffc8068ae1f320  7       680       6ff     32 Private      READWRITE
ffffc80689b290b0  9       700       701      2 Private      READWRITE
ffffc80688da04f0  8       710       711      2 Private      READWRITE
ffffc80682795760  6       720       723      0 Mapped       READONLY
Pagefile section, shared commit 0x4
ffffc80688d85670 10       730       731      0 Mapped       READONLY
Pagefile section, shared commit 0x2
ffffc80689bdd9e0  9       740       741      2 Private      READWRITE
ffffc80688da57b0  8       750       755      0 Mapped       READONLY
\Windows\en-US\explorer.exe.mui
```

...
Total VADs: 574, average level: 8, maximum depth: 10
Total private commit: 0x3420 pages (53376 KB)
Total shared commit: 0x478 pages (4576 KB)

Rotate VADs

A video card driver must typically copy data from the user-mode graphics application to various other system memory, including the video card memory and the AGP port's memory, both of which have different caching attributes as well as addresses. To quickly allow these different views of memory to be mapped into a process, and to support the different cache attributes, the memory manager implements rotate VADs, which allow video drivers to transfer data directly by using the GPU and to rotate unneeded memory in and out of the process view pages on demand. **Figure 5-30** shows an example of how the same virtual address can rotate between video RAM and virtual memory.

FIGURE 5-30 Rotate VADs.

NUMA

Each new release of Windows provides new enhancements to the memory manager to make better use of non-uniform memory architecture (NUMA) machines, such as large server systems as well as Intel i7 and AMD Opteron SMP workstations. The NUMA support in the memory manager adds intelligent knowledge of node information such as location, topology, and access costs to allow applications and drivers to take advantage of NUMA capabilities, while abstracting the underlying hardware details.

When the memory manager is initializing, it calls the `MiComputeNumaCosts` function to perform various page and cache operations on different nodes. It then computes the time it took for those operations to complete. Based on this information, it builds a node graph of access costs (the distance between a node and any other node on the system). When the system requires pages for a given operation, it consults the graph to choose the most optimal node (that is, the closest). If no memory is available on that node, it chooses the next closest node, and so on.

Although the memory manager ensures that, whenever possible, memory allocations come from the ideal processor's node (the *ideal node*) of the thread making the allocation, it also provides functions that allow applications to choose their own node, such as the `VirtualAllocExNuma`, `CreateFileMappingNuma`, `MapViewOfFileExNuma`, and `AllocateUserPhysicalPagesNuma` APIs.

The ideal node isn't used only when applications allocate memory but also during kernel operation and page faults. For example, when a thread running on a non-ideal processor takes a page fault, the memory manager won't use the current node. Instead, it will allocate memory from the thread's ideal node. Although this might result in slower access time while the thread is still running on this CPU, overall memory access will be optimized as the thread migrates back to its ideal node. In any case, if the ideal node is out of resources, the closest node to the ideal node is chosen and not a random other node. Just like user-mode

applications, however, drivers can specify their own node when using APIs such as `MmAllocatePagesForMdlEx` or `MmAllocateContiguousMemorySpecifyCacheNode`.

Various memory manager pools and data structures are also optimized to take advantage of NUMA nodes. The memory manager tries to evenly use physical memory from all the nodes on the system to hold the non-paged pool. When a non-paged pool allocation is made, the memory manager uses the ideal node as an index to choose a virtual memory address range inside non-paged pool that corresponds to physical memory belonging to this node. In addition, per-NUMA node pool free lists are created to efficiently leverage these types of memory configurations. Apart from non-paged pool, the system cache and system PTEs are also similarly allocated across all nodes, as well as the memory manager's look-aside lists.

Finally, when the system needs to zero pages, it does so in parallel across different NUMA nodes by creating threads with NUMA affinities that correspond to the nodes in which the physical memory is located. The logical prefetcher and SuperFetch (described in the section "Proactive memory management [SuperFetch]") also use the ideal node of the target process when prefetching, while soft page faults cause pages to migrate to the ideal node of the faulting thread.

Section objects

As noted earlier in this chapter in the "**Shared memory and mapped files**" section, the *section object*, which the Windows subsystem calls a *file mapping object*, represents a block of memory that two or more processes can share. A section object can be mapped to the paging file or to another file on disk.

The executive uses sections to load executable images into memory, and the cache manager uses them to access data in a cached file. (See Chapter 14 in Part 2 for more information on how the cache manager uses section objects.) You can also use section objects to map a file into a process address space. The file can then be accessed as a large array

by mapping different views of the section object and reading or writing to memory rather than to the file—an activity called *mapped file I/O*. When the program accesses an invalid page (one not in physical memory), a page fault occurs and the memory manager automatically brings the page into memory from the mapped file or page file. If the application modifies the page, the memory manager writes the changes back to the file during its normal paging operations. (Alternatively, the application can flush a view explicitly by using the Windows `FlushViewOfFile` function.)

Like other objects, section objects are allocated and deallocated by the object manager. The object manager creates and initializes an object header, which it uses to manage the objects; the memory manager defines the body of the section object. (See Chapter 8 in Part 2 for more on the object manager). The memory manager also implements services that user-mode threads can call to retrieve and change the attributes stored in the body of section objects. The structure of a section object is shown in **Figure 5-31**. **Table 5-14** summarizes the unique attributes stored in section objects.

FIGURE 5-31 A section object.

TABLE 5-14 Section object body attributes

EXPERIMENT: VIEWING SECTION OBJECTS

You can use Process Explorer from Sysinternals to see files mapped by a process. Follow these steps:

1. Open the **View** menu, choose **Lower Pane View**, and select **DLLs**.

2. Open the **View** menu, choose **Select Columns**, choose **DLL**, and enable the **Mapping Type** column.

3. Notice the files marked as Data in the Mapping column. These are mapped files rather than DLLs and other files the image loader loads as modules. Section objects that are backed by a page file are indicated in the Name column as <Pagefile Backed>. Otherwise, the file name is shown.

Another way to view section objects is to switch to handle view (open the **View** menu, choose **Lower Pane View**, and select **Handles**) and look for objects of type Section. In the following screenshot, the object name (if it exists) is shown. This is not the file name backing the section (if any); it's the name given to the section in the object manager's namespace. (See Chapter 8 in Part 2 for more on the object manager.) Double-clicking the entry shows more information on the object, such as the number of open handles and its security descriptor.

The data structures maintained by the memory manager that describe mapped sections are shown in **Figure 5-32**. These structures ensure that data read from mapped files is consistent regardless of the type of access (open file, mapped file, and so on). For each open file (represented by a file object), there is a single section object pointers structure. This structure is the key to maintaining data consistency for all types of file access as well as to providing caching for files. The section object pointers structure points to one or two control areas. One control area is used to map the file when it is accessed as a data file and the other is used to map the file when it is run as an executable image. A control area in turn points to subsection structures that describe the mapping information for each section of the file (read-only, read/write, copy-on-write, and so on). The control area also points to a segment structure allocated in paged pool, which in turn points to the prototype PTEs used to map to the actual pages mapped by the section object. As described earlier in this chapter, process page tables point to these prototype PTEs, which in turn map the pages being referenced.

FIGURE 5-32 Internal section structures.

Although Windows ensures that any process that accesses (reads or writes) a file will always see the same consistent data, there is one case in which two copies of pages of a file can reside in physical memory. (Even in this case, all accessors get the latest copy and data consistency is maintained.) This duplication can happen when an image file has been accessed as a data file (having been read or written) and is then run as an executable image. (An example might be when an image is linked and then run; the linker had the file open for data access, and then when the image was run, the image loader mapped it as an executable.) Internally, the following actions occur:

1. If the executable file was created using the file-mapping APIs or the cache manager, a data control area is created to represent the data pages in the image file being read or written.

2. When the image is run and the section object is created to map the image as an executable, the memory manager finds that the section object pointers for the image file point to a data control area and flushes the section. This step is necessary to ensure that any modified pages have been written to disk before accessing the image through the image control area.

3. The memory manager creates a control area for the image file.

4. As the image begins execution, its (read-only) pages are faulted in from the image file or copied directly over from the data file if the corresponding data page is resident.

Because the pages mapped by the data control area might still be resident (on the standby list), this is the one case in which two copies of the same data are in two different pages in memory. However, this duplication doesn't result in a data consistency issue. This is because, as mentioned, the data control area has already been flushed to disk, so the pages read from the image are up to date (and these pages are never written back to disk).

EXPERIMENT: VIEWING CONTROL AREAS

To find the address of the control area structures for a file, you must first get the address of the file object in question. You can obtain this address through the kernel debugger by dumping the process handle table with the `!handle` command and noting the object address of a file object. Although the kernel debugger `!file` command displays the basic information in a file object, it doesn't display the pointer to the section object pointers structure. Then, using the `dt` command, format the file object to get the address of the section object pointers structure. This structure consists of three pointers: a pointer to the data control area, a pointer to the shared cache map (explained in Chapter 14 in Part 2), and a pointer to the image control area. From the section object pointers structure, you can obtain the address of a control area for the file (if one exists) and feed that address into the `!ca` command.

For example, if you open a PowerPoint file and use `!handle` to display the handle table for that process, you will find an open handle to the PowerPoint file (you can do a text search). (For more information on using `!handle`, see the "Object manager" section in Chapter 8 in Part 2 or the debugger documentation.)

Click here to view code image

```
lkd> !process 0 0 powerpnt.exe
PROCESS ffffc8068913e080
    SessionId: 1  Cid: 2b64    Peb: 01249000  ParentCid: 1d38
    DirBase: 252e25000  ObjectTable: ffffda8f49269c40  HandleCount:
1915.
    Image: POWERPNT.EXE
lkd> .process /p ffffc8068913e080
Implicit process is now ffffc806'8913e080
lkd> !handle
...
0c08: Object: ffffc8068f56a630  GrantedAccess: 00120089 Entry:
ffffda8f491d0020
Object: ffffc8068f56a630  Type: (ffffc8068256cb00) File
```

```
    ObjectHeader: ffffc8068f56a600 (new version)
        HandleCount: 1  PointerCount: 30839
        Directory Object: 00000000  Name:
\WindowsInternals\7thEdition\Chapter05\
diagrams.pptx {HarddiskVolume2}
...
```

Taking the file object address (FFFFC8068F56A630) and formatting it with `dt` results in this:

Click here to view code image

```
lkd> dt nt!_file_object ffffc8068f56a630
   +0x000 Type           : 0n5
   +0x002 Size           : 0n216
   +0x008 DeviceObject   : 0xffffc806'8408cb40 _DEVICE_OBJECT
   +0x010 Vpb            : 0xffffc806'82feba00 _VPB
   +0x018 FsContext      : 0xffffda8f'5137cbd0 Void
   +0x020 FsContext2     : 0xffffda8f'4366d590 Void
   +0x028 SectionObjectPointer : 0xffffc806'8ec0c558
_SECTION_OBJECT_POINTERS
...
```

Taking the address of the section object pointers structure and formatting it with `dt` results in this:

Click here to view code image

```
lkd> dt nt!_section_object_pointers 0xffffc806'8ec0c558
   +0x000 DataSectionObject : 0xffffc806'8e838c10 Void
   +0x008 SharedCacheMap   : 0xffffc806'8d967bd0 Void
   +0x010 ImageSectionObject : (null)
```

Finally, you can use `!ca` to display the control area using the address:

Click here to view code image

```
lkd> !ca 0xffffc806'8e838c10
ControlArea  @ ffffc8068e838c10
  Segment      ffffda8f4d97fdc0  Flink      ffffc8068ecf97b8  Blink
ffffc8068ecf97b8
  Section Ref              1  Pfn Ref              58  Mapped Views        2
  User Ref                 0  WaitForDel            0  Flush Count         0
  File Object  ffffc8068e5d3d50  ModWriteCount         0  System
Views         2
  WritableRefs             0
  Flags (8080) File
WasPurged   \WindowsInternalsBook\7thEdition\Chapter05\diagrams.pptx

Segment @ ffffda8f4d97fdc0
  ControlArea     ffffc8068e838c10  ExtendInfo    0000000000000000
  Total Ptes               80
  Segment Size             80000  Committed              0
  Flags (c0000) ProtectionMask

Subsection 1 @ ffffc8068e838c90
  ControlArea  ffffc8068e838c10  Starting Sector       0  Number Of
Sectors   58
  Base Pte     ffffda8f48eb6d40  Ptes In Subsect       58  Unused Ptes         0
  Flags                  d  Sector Offset         0  Protection          6
  Accessed
  Flink       ffffc8068bb7fcf0  Blink   ffffc8068bb7fcf0  MappedViews         2

Subsection 2 @ ffffc8068c2e05b0
  ControlArea  ffffc8068e838c10  Starting Sector       58  Number Of
Sectors   28
  Base Pte     ffffda8f3cc45000  Ptes In Subsect       28  Unused
Ptes       1d8
  Flags                  d  Sector Offset         0  Protection          6
  Accessed
  Flink       ffffc8068c2e0600  Blink   ffffc8068c2e0600  MappedViews         1
```

Another technique is to display the list of all control areas with the `!memusage` command. The following excerpt is from the output of this command. (The command might take a long time to complete on a system with a large amount of memory.)

Click here to view code image

```
lkd> !memusage
 loading PFN database
loading (100% complete)

Compiling memory usage data (99% Complete).

           Zeroed:    98533 (  394132 kb)
             Free:     1405 (    5620 kb)
          Standby:   331221 ( 1324884 kb)
         Modified:    83806 (  335224 kb)
   ModifiedNoWrite:      116 (     464 kb)
      Active/Valid:  1556154 ( 6224616 kb)
       Transition:        5 (      20 kb)
        SLIST/Bad:     1614 (    6456 kb)
          Unknown:        0 (       0 kb)
            TOTAL:  2072854 ( 8291416 kb)

Dangling Yes Commit:      130 (     520 kb)
 Dangling No Commit:   514812 ( 2059248 kb)
  Building kernel map
  Finished building kernel map

 (Master1 0 for 1c0)

 (Master1 0 for e80)

 (Master1 0 for ec0)

 (Master1 0 for f00)
Scanning PFN database - (02% complete)
```

(Master1 0 for de80)
Scanning PFN database - (100% complete)

Usage Summary (in Kb):
Control Valid Standby Dirty Shared Locked PageTables name
ffffffffd 1684540 0 0 0 1684540 0 AWE
ffff8c0b7e4797d0 64 0 0 0 0 0 mapped_file(Microsoft-Windows-Kernel-PnP%4Configuration.evtx)
ffff8c0b7e481650 0 4 0 0 0 0 mapped_file(No name for file)
ffff8c0b7e493c00 0 40 0 0 0 0 mapped_file(FSD-{ED5680AF-
0543-4367-A331-850F30190B44}.FSD)
ffff8c0b7e4a1b30 8 12 0 0 0 0 mapped_file(msidle.dll)
ffff8c0b7e4a7c40 128 0 0 0 0 0 mapped_file(Microsoft-Windows-Diagnosis-PCW%4Operational.evtx)
ffff8c0b7e4a9010 16 8 0 16 0 0 mapped_file(netjoin.dll)8a04db00 ...
ffff8c0b7f8cc360 8212 0 0 0 0 0 mapped_file(OUTLOOK.EXE)
ffff8c0b7f8cd1a0 52 28 0 0 0 0 mapped_file(verdanab.ttf)
ffff8c0b7f8ce910 0 4 0 0 0 0 mapped_file(No name for file)
ffff8c0b7f8d3590 0 4 0 0 0 0 mapped_file(No name for file)
...

The `Control` column points to the control area structure that describes the mapped file. You can display control areas, segments, and subsections with the kernel debugger `!ca` command. For example, to dump the control area for the mapped file Outlook.exe in this example, type the `!ca` command followed by the `Control` column number, as shown here:

Click here to view code image

```
lkd> !ca ffff8c0b7f8cc360

ControlArea  @ ffff8c0b7f8cc360
  Segment      ffffdf08d8a55670  Flink      ffff8c0b834f1fd0  Blink
ffff8c0b834f1fd0
  Section Ref              1  Pfn Ref             806  Mapped Views          1
  User Ref                 2  WaitForDel           0  Flush Count        c5a0
  File Object  ffff8c0b7f0e94e0  ModWriteCount          0  System
Views        ffff
  WritableRefs        80000161
  Flags (a0) Image File

      \Program Files (x86)\Microsoft Office\root\Office16\OUTLOOK.EXE

Segment @ ffffdf08d8a55670
  ControlArea     ffff8c0b7f8cc360  BasedAddress  0000000000be0000
  Total Ptes               1609
  Segment Size            1609000  Committed                0
  Image Commit               f4  Image Info    ffffdf08d8a556b8
  ProtoPtes       ffffdf08dab6b000
  Flags (c20000) ProtectionMask

Subsection 1 @ ffff8c0b7f8cc3e0
  ControlArea  ffff8c0b7f8cc360  Starting Sector        0  Number Of
Sectors    2
  Base Pte     ffffdf08dab6b000  Ptes In Subsect        1  Unused Ptes          0
  Flags                    2  Sector Offset          0  Protection           1

Subsection 2 @ ffff8c0b7f8cc418
  ControlArea  ffff8c0b7f8cc360  Starting Sector        2  Number Of
Sectors 7b17
  Base Pte     ffffdf08dab6b008  Ptes In Subsect      f63  Unused Ptes          0
  Flags                    6  Sector Offset          0  Protection           3
```

Subsection 3 @ ffff8c0b7f8cc450
ControlArea ffff8c0b7f8cc360 Starting Sector 7b19 Number Of Sectors 19a4
Base Pte ffffdf08dab72b20 Ptes In Subsect 335 Unused Ptes 0
Flags 2 Sector Offset 0 Protection 1

Subsection 4 @ ffff8c0b7f8cc488
ControlArea ffff8c0b7f8cc360 Starting Sector 94bd Number Of Sectors 764
Base Pte ffffdf08dab744c8 Ptes In Subsect f2 Unused Ptes 0
Flags a Sector Offset 0 Protection 5

Subsection 5 @ ffff8c0b7f8cc4c0
ControlArea ffff8c0b7f8cc360 Starting Sector 9c21 Number Of Sectors 1
Base Pte ffffdf08dab74c58 Ptes In Subsect 1 Unused Ptes 0
Flags a Sector Offset 0 Protection 5

Subsection 6 @ ffff8c0b7f8cc4f8
ControlArea ffff8c0b7f8cc360 Starting Sector 9c22 Number Of Sectors 1
Base Pte ffffdf08dab74c60 Ptes In Subsect 1 Unused Ptes 0
Flags a Sector Offset 0 Protection 5

Subsection 7 @ ffff8c0b7f8cc530
ControlArea ffff8c0b7f8cc360 Starting Sector 9c23 Number Of Sectors c62
Base Pte ffffdf08dab74c68 Ptes In Subsect 18d Unused Ptes 0
Flags 2 Sector Offset 0 Protection 1

Subsection 8 @ ffff8c0b7f8cc568
ControlArea ffff8c0b7f8cc360 Starting Sector a885 Number Of Sectors 771
Base Pte ffffdf08dab758d0 Ptes In Subsect ef Unused Ptes 0
Flags 2 Sector Offset 0 Protection 1

Working sets

Now that you've looked at how Windows keeps track of physical memory and how much memory it can support, we'll explain how Windows keeps a subset of virtual addresses in physical memory.

As you'll recall, a subset of virtual pages resident in physical memory is called a *working set*. There are three kinds of working sets:

- **Process working sets** These contain the pages referenced by threads within a single process.

- **System working sets** These contain the resident subset of the pageable system code (for example, Ntoskrnl.exe and drivers), paged pool, and the system cache.

- **Session's working set** Each session has a working set that contains the resident subset of the kernel-mode session-specific data structures allocated by the kernel-mode part of the Windows subsystem (Win32k.sys), session paged pool, session mapped views, and other session-space device drivers.

Before examining the details of each type of working set, let's look at the overall policy for deciding which pages are brought into physical memory and how long they remain.

Demand paging

The Windows memory manager uses a demand-paging algorithm with clustering to load pages into memory. When a thread receives a page fault, the memory manager loads into memory the faulted page plus a small number of pages preceding and/or following it. This strategy attempts to minimize the number of paging I/Os a thread will incur. Because programs—especially large ones—tend to execute in small regions of their address space at any given time, loading clusters of virtual pages reduces the number of disk reads. For page faults that reference data pages in images, the cluster size is three pages. For all other page faults, the cluster size is seven pages.

However, a demand-paging policy can result in a process incurring many page faults when its threads first begin executing or when they resume execution at a later point. To optimize the startup of a process (and the system), Windows has an intelligent prefetch engine called the *logical prefetcher*, described in the next section. Further optimization and prefetching is performed by another component, called *SuperFetch*, described later in the chapter.

Logical prefetcher and ReadyBoot

During a typical system boot or application startup, the order of faults is such that some pages are brought in from one part of a file, then perhaps from a distant part of the same file, then from a different file, then perhaps from a directory, and then again from the first file. This jumping around slows down each access considerably. Indeed, analysis shows that disk seek times are a dominant factor in slowing boot and application startup times. By prefetching batches of pages all at once, you can achieve a more sensible ordering of access without excessive backtracking, thus improving the overall time for system and application startup. The pages that are needed can be known in advance because of the high correlation in accesses across boots or application starts.

The prefetcher tries to speed the boot process and application startup by monitoring the data and code accessed by boot and application startups and using that information at the beginning of a subsequent boot or application startup to read in the code and data. When the prefetcher is active, the memory manager notifies the prefetcher code in the kernel of page faults—those that require that data be read from disk (hard faults) and those that simply require data already in memory to be added to a process's working set (soft faults). The prefetcher monitors the first 10 seconds of application startup. For boot, the prefetcher by default traces from system start through the 30 seconds following the start of the user's shell (typically Explorer) or, failing that, through 60 seconds following Windows service initialization or through 120 seconds, whichever comes first.

The trace assembled in the kernel notes faults taken on the NTFS master file table (MFT) metadata file (if the application accesses files or directories on NTFS volumes), referenced files, and referenced directories. With the trace assembled, the kernel prefetcher code waits for requests from the prefetcher component of the `Superfetch` service (%SystemRoot%\System32\Sysmain.dll), running in an instance of Svchost. The `Superfetch` service is responsible for both the logical prefetching component in the kernel and the SuperFetch component that we'll talk about later. The prefetcher signals the `\KernelObjects\PrefetchTracesReady` event to inform the `Superfetch` service that it can now query trace data.

NOTE

You can enable or disable prefetching of the boot or application startups by editing the DWORD registry value `EnablePrefetcher` in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters key. Set it to `0` to disable prefetching altogether, `1` to enable prefetching of only applications, `2` for prefetching of boot only, and `3` for both boot and applications.

The `Superfetch` service (which hosts the logical prefetcher, although it is a completely separate component from the actual SuperFetch functionality) performs a call to the internal `NtQuerySystemInformation` system call requesting the trace data. The logical prefetcher post-processes the trace data, combining it with previously collected data, and writes it to a file in the %SystemRoot%\Prefetch folder. (See **Figure 5-33**.) The file's name is the name of the application to which the trace applies followed by a dash and the hexadecimal representation of a hash of the file's path. The file has a .pf extension. An example would be NOTEPAD.EXE-9FB27C0E.PF.

FIGURE 5-33 Prefetch folder.

There is an exception to the file name rule for images that host other components, including the Microsoft Management Console (%SystemRoot%\System32\Mmc.exe), the service hosting process (%SystemRoot%\System32\Svchost.exe), the RunDLL component (%SystemRoot%\System32\Rundll32.exe), and Dllhost (%SystemRoot%\System32\Dllhost.exe). Because add-on components are specified on the command line for these applications, the prefetcher includes the command line in the generated hash. Thus, invocations of these applications with different components on the command line will result in different traces.

For a system boot, a different mechanism is used, called *ReadyBoot*. ReadyBoot tries to optimize I/O operations by creating large and efficient I/O reads and storing the data in RAM. When system components require the data, it's serviced through the stored RAM. This especially benefits mechanical disks, but can be also useful for SSDs. Information on the files to prefetch is stored after boot in the ReadyBoot subdirectory of the Prefetch directory shown in **Figure 5-33**. Once boot is com-

plete, the cached data in RAM is deleted. For very fast SSDs, ReadyBoot is off by default because its gains are marginal, if any.

When the system boots or an application starts, the prefetcher is called to give it an opportunity to prefetch. The prefetcher looks in the prefetch directory to see if a trace file exists for the prefetch scenario in question. If it does, the prefetcher calls NTFS to prefetch any MFT metadata file references, reads in the contents of each of the directories referenced, and finally opens each file referenced. It then calls the memory manager function `MmPrefetchPages` to read in any data and code specified in the trace that's not already in memory. The memory manager initiates all the reads asynchronously and then waits for them to complete before letting an application's startup continue.

EXPERIMENT: WATCHING PREFETCH FILE READS AND WRITES

If you capture a trace of application startup with Process Monitor from Sysinternals on a client edition of Windows (Windows Server editions disable prefetching by default), you can see the prefetcher check for and read the application's prefetch file (if it exists). In addition, you can see the prefetcher write out a new copy of the file roughly 10 seconds after the application starts. Here is a capture of Notepad startup with an `Include` filter set to `prefetch` so that Process Monitor shows only accesses to the %SystemRoot%\Prefetch directory:

Lines 0–3 show the Notepad prefetch file being read in the context of the Notepad process during its startup. Lines 7–19 (which have time stamps 10 seconds later than the first four lines) show the `Superfetch` service—running in the context of a Svchost process—writing out the updated prefetch file.

To minimize seeking even further, every three days or so, during system idle periods, the `Superfetch` service organizes a list of files and directories in the order that they are referenced during a boot or application start and stores it in a file named %SystemRoot%\Prefetch\Layout.ini (see **Figure 5-34**). This list also includes frequently accessed files tracked by `Superfetch`.

FIGURE 5-34 Prefetch defragmentation layout file.

It then launches the system defragmenter with a command-line option that tells the defragmenter to defragment based on the contents of the file instead of performing a full defrag. The defragmenter finds a contiguous area on each volume large enough to hold all the listed files and directories that reside on that volume and then moves them in their entirety into that area so that they are stored one after the other. Thus, future prefetch operations will even be more efficient because all the data read in is now stored physically on the disk in the order it will be read. Because the files defragmented for prefetching usually number only in the hundreds, this defragmentation is much faster than full-volume defragmentations.

Placement policy

When a thread receives a page fault, the memory manager must determine where in physical memory to put the virtual page. The set of rules it uses to determine the best position is called a *placement policy*. Windows considers the size of CPU memory caches when choosing page frames to minimize unnecessary thrashing of the cache.

If physical memory is full when a page fault occurs, Windows uses a replacement policy to determine which virtual page must be removed from memory to make room for the new page. Common replacement policies include least recently used (LRU) and first in, first out (FIFO). The LRU algorithm (also known as the *clock algorithm*, as implemented in most versions of UNIX) requires the virtual memory system to track when a page in memory is used. When a new page frame is required, the page that hasn't been used for the greatest amount of time is removed from the working set. The FIFO algorithm is somewhat simpler: It removes the page that has been in physical memory for the greatest amount of time, regardless of how often it's been used.

Replacement policies can be further characterized as either global or local. A global replacement policy allows a page fault to be satisfied by any page frame, regardless of whether that frame is owned by another process. For example, a global replacement policy using the FIFO algorithm would locate the page that has been in memory the longest and would free it to satisfy a page fault. A local replacement policy would limit its search for the oldest page to the set of pages already owned by the process that incurred the page fault. Be aware that global replacement policies make processes vulnerable to the behavior of other processes. For example, an ill-behaved application can undermine the entire operating system by inducing excessive paging activity in all processes.

Windows implements a combination of local and global replacement policies. When a working set reaches its limit and/or needs to be trimmed because of demands for physical memory, the memory man-

ager removes pages from working sets until it has determined there are enough free pages.